COMPUTER VISION

LECTURE NOTES

IMAGE FORMATION
GEOMETRY
RECOGNITION
PERCEPTION
APPLIED VISION



NOTES PREPARED BY

Matt Clifford Trinidad Ramos

Contents

Lecture 1: Introduction: Scope, Challenges, and Applications 1

1 Computer Vision: Purpose and Scope 2
- 1.1 What Computer Vision Tries to Recover . . . 2
- 1.2 Why Visual Data Matter . . . 3
- 1.3 Vision and Graphics . . . 3

2 Why Vision Is Difficult 4
- 2.1 An Ill-Posed Inverse Problem . . . 4
- 2.2 Nuisance Parameters . . . 4
- 2.3 Context and Intra-Class Variation . . . 5
- 2.4 Visual Cues . . . 5
- 2.5 Scale and Complexity . . . 6

3 Historical Development and Changing Emphasis 6

4 Measurement and Semantic Understanding 8
- 4.1 Vision as a Measurement Device . . . 8
- 4.2 Vision as a Source of Semantic Information . . . 8

5 Core Areas of Computer Vision 9
- 5.1 Early Vision: Image Formation and Processing . . . 9
- 5.2 Mid-Level Vision: Grouping, Fitting, and Tracking . . . 10
- 5.3 Multi-View Geometry . . . 10
- 5.4 Recognition . . . 11
- 5.5 High-Level Geometric Vision . . . 11
- 5.6 Advanced Topics . . . 11
- 5.7 Image Collections and Image-Based Rendering . . . 11

6 Applications of Computer Vision 12

7 Computer Vision and Machine Learning 15
- 7.1 The Role of Machine Learning . . . 15
- 7.2 Related Disciplines . . . 16
- 7.3 Active Research Topics . . . 16
- 7.4 Research Publications . . . 16
- 7.5 Further Reading . . . 16

Lecture 1 Quick Review 17

Lecture 2: Image Acquisition and Digitalization **18**

8 Image Formation and Camera Acquisition **19**
8.1 Illumination and Surface Properties . . . 19
8.2 A Spectral Image-Formation Model . . . 19
8.3 Perspective and Orthographic Projection . . . 20
8.4 Color Sensors . . . 21

9 Digital Images: Sampling, Quantization, and Resolution **21**
9.1 From a Continuous Image to a Digital Image . . . 21
9.2 Quantization and Bit Depth . . . 22
9.3 Digital Image Types . . . 23
9.4 Pixels, Memory, and Spatial Resolution . . . 23

10 Pixel Relationships and Image Resampling **25**
10.1 Neighborhoods . . . 25
10.2 Connectivity, Adjacency, and Paths . . . 25
10.3 Distance Measures . . . 26
10.4 Interpolation . . . 26

Lecture 2 Quick Review **29**

Lecture 3: Color, Perception, and Texture **30**

11 Color: Light and Human Vision **31**
11.1 The Nature of Color . . . 31
11.2 The Eye as a Sensing System . . . 31
11.3 Spectral Filtering and Metamers . . . 33

12 Color Matching and Color Spaces **34**
12.1 Trichromacy and Color Matching . . . 34
12.2 Grassmann's Laws . . . 34
12.3 Color-Matching Functions as a Linear Transform . . . 35
12.4 RGB and CIE XYZ . . . 35
12.5 Toward Perceptual Uniformity . . . 35
12.6 HSV . . . 36

13 Color Constancy and White Balance **37**
13.1 Constancy and Context . . . 37
13.2 Chromatic Adaptation . . . 38
13.3 White Balance . . . 39

14 Color in Computer Vision and Texture **40**
14.1 Uses of Color . . . 40
14.2 Structural and Statistical Texture . . . 40
14.3 Local Binary Patterns . . . 41
14.4 Image Data Structure . . . 41

Lecture 3 Quick Review **42**

Lecture 4: Histograms and Intensity Transformations 43

15 Histograms and Point Transformations 44
15.1 Intensity Histograms . . . 44
15.2 Histograms of Color Images . . . 45
15.3 Point Transformations . . . 45

16 Histogram Equalization, Projection, and Plateau Methods 46
16.1 Histogram Equalization . . . 46
16.2 Histogram Projection . . . 46
16.3 Plateau Equalization . . . 47

17 Histogram Specification and Contrast Stretching 48
17.1 Histogram Specification . . . 48
17.2 Contrast Stretching . . . 48

18 Applications of Histogram Processing 49
18.1 Thresholding . . . 49
18.2 Normalization and Histogram Matching . . . 49
18.3 Medical-Image Examples . . . 49
18.4 High-Dynamic-Range Display . . . 50

Lecture 4 Quick Review 51

Lecture 5: Spatial Filters and Image Enhancement 52

19 Spatial Filtering Foundations 53
19.1 Neighborhood Operations . . . 53
19.2 The Spatial Filtering Process . . . 53
19.3 Correlation and Convolution . . . 54
19.4 Image Borders . . . 54

20 Smoothing Spatial Filters 54
20.1 Averaging Filters . . . 54
20.2 Weighted Smoothing . . . 55
20.3 Smoothing as Preprocessing . . . 55
20.4 Averaging and Median Filtering . . . 55

21 Sharpening and Spatial Derivatives 56
21.1 First and Second Discrete Derivatives . . . 56
21.2 The Laplacian . . . 56
21.3 Unsharp Masking and Highboost Filtering . . . 57
21.4 First-Derivative Filtering and the Gradient . . . 58
21.5 Sobel Operators . . . 58

22 Combining Spatial Enhancement Methods 59
22.1 Bone-Scan Enhancement Pipeline . . . 59

Lecture 5 Quick Review 61

Lecture 6: Edges and the Canny Detector 62

23 Edge Detection Fundamentals 63
23.1 Purpose and Origin of Edges . . . 63
23.2 Image Gradient and Edge Strength . . . 63
23.3 Finite-Difference Filters . . . 64
23.4 Noise and the Need to Smooth . . . 64
23.5 Derivative Theorem of Convolution . . . 65
23.6 Smoothing Scale and Localization . . . 65

24 Localizing and Linking Edge Curves 66
24.1 Why Gradient Ridges Must Be Thinned . . . 66
24.2 Non-Maximum Suppression . . . 66
24.3 Predicting the Next Edge Point . . . 66
24.4 Criteria for an Optimal Edge Detector . . . 66

25 The Canny Edge Detector 67
25.1 Model and Motivation . . . 67
25.2 Algorithm . . . 67
25.3 Hysteresis Thresholding . . . 67
25.4 Effect of Gaussian Scale . . . 67
25.5 Edges as an Intermediate Representation . . . 68

Lecture 6 Quick Review 69

Lecture 7: Corners, Blobs, and Invariant Features 70

26 Feature Extraction and Harris Corners 71
26.1 Why Extract Features? . . . 71
26.2 Corner Intuition . . . 71
26.3 PCA View of Local Gradient Distributions . . . 71
26.4 Harris Change Function . . . 72
26.5 Interpreting the Second Moment Matrix . . . 72
26.6 Harris Corner Response and Algorithm . . . 73
26.7 Invariance Properties . . . 73

27 Scale-Invariant Blob Detection 74
27.1 From Edges to Blobs . . . 74
27.2 Why Scale Normalization Is Needed . . . 74
27.3 Characteristic Scale . . . 75
27.4 Scale-Space Blob Detector . . . 75
27.5 Difference of Gaussians . . . 76

28 Affine Adaptation and Invariant Description 76
28.1 From Scale Invariance to Affine Invariance . . . 76
28.2 Orientation Ambiguity . . . 76
28.3 Iterative Affine Adaptation . . . 77
28.4 Feature-Extraction Pipeline . . . 77
28.5 Invariance and Covariance . . . 77

Lecture 7 Quick Review 79

Lecture 8: Fitting 80

29 Model Fitting and the Hough Transform 81
29.1 What Fitting Tries to Solve . . . 81
29.2 Voting Schemes . . . 81
29.3 Line Parameterization . . . 81
29.4 Hough Algorithm for Lines . . . 82
29.5 Practical Details, Strengths, and Limitations . . . 82

30 Least Squares and Robust Fitting 83
30.1 Vertical Least-Squares Line Fitting . . . 83
30.2 Total Least Squares . . . 83
30.3 General Curves and Conics . . . 85
30.4 Why Ordinary Least Squares Is Not Robust . . . 86

31 RANSAC and Multiple-Model Fitting 87
31.1 Random Sample Consensus . . . 87
31.2 Choosing RANSAC Parameters . . . 87
31.3 RANSAC Strengths and Limitations . . . 88
31.4 Incremental Line Fitting . . . 88
31.5 K-Lines . . . 89

32 Model Complexity and Selection 89
32.1 Bias–Variance Tradeoff . . . 89
32.2 Occam's Razor . . . 90

Lecture 8 Quick Review 91

Lecture 9: Image Alignment 92

33 Image Alignment and 2D Transformations 93
33.1 Motivation and Challenges . . . 93
33.2 Alignment as a Fitting Problem . . . 93
33.3 Transformation Families . . . 93
33.4 Fitting an Affine Transformation . . . 94

34 Feature Descriptors and Matching 95
34.1 Patch Descriptors . . . 95
34.2 SIFT Descriptor . . . 96
34.3 Generating Putative Matches . . . 96

35 Robust Alignment from Putative Matches 97
35.1 The Outlier Problem . . . 97
35.2 RANSAC for Alignment . . . 97
35.3 Incremental Alignment . . . 97
35.4 Hough-Space Alignment . . . 98
35.5 Lowe's Hough-Space Verification . . . 98

36 Homographies and Panorama Stitching 98
36.1 Beyond Affine Transformations . . . 98
36.2 Panorama Recognition and Stitching . . . 99
36.3 Pyramid Blending . . . 99
36.4 Alpha Blending . . . 100
36.5 Gain Compensation . . . 100
36.6 Design Choices in Alignment Applications . . . 100

Lecture 9 Quick Review 102

LECTURE 1

Introduction: Scope, Challenges, and Applications

From visual measurements to descriptions of the world.

Scope · Visual cues · Geometry · Recognition

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 1

Computer Vision: Purpose and Scope

Computer vision aims to make computational systems extract properties of the three-dimensional world from one or more images. The desired result may be a physical measurement, an object or scene category, a location, a spatial relation, or another description that is useful for a task.



Figure 1.1. A visual metaphor for the central goal of computer vision: enabling computational systems to interpret visual input.



Figure 1.2. The source's computer-vision pipeline: visual input becomes an interpretation of the world through sensing and interpretation.

1.1 What Computer Vision Tries to Recover

> **Computer Vision**
>
> Given one image or several images, computer vision extracts properties of the three-dimensional world represented by those images.

A traffic scene illustrates the range of possible outputs. From the same image, a system may be asked to determine the number and type of vehicles, the location of the closest obstacle, the level of congestion, or

the location and nature of the observed scene.

> **Scene understanding from a single photograph**
>
> A visual scene may support several simultaneous questions: where the image was captured, how many people are present, what they are doing, what objects are supporting or surrounding them, and whether the situation contains an unusual or humorous interaction. These questions require more than measuring isolated pixels.

1.2 Why Visual Data Matter

Images and video occur in personal photo collections, movies, news, sport, surveillance and security, medical imaging, scientific imaging, and many other settings. Their abundance makes visual analysis useful in both everyday systems and specialized scientific applications.

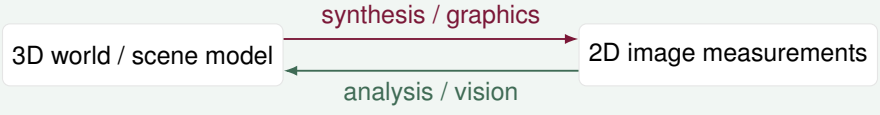
The broad motivation for computer vision includes safety, health technology, security, comfort, entertainment, and accessibility. The same field therefore spans both measurement-oriented problems and semantic interpretation.



Figure 1.3. Representative domains in which images and video occur: personal media, movies and news, surveillance, and scientific or medical imaging.

1.3 Vision and Graphics

Computer graphics and computer vision address opposite directions of the same relationship. Graphics synthesizes images from a model of the world; vision analyzes images to infer properties of the world.

> **Vision and graphics: opposite directions**
>
> 
>

The central computer-vision problem is inverse: a three-dimensional world produces a two-dimensional image, but the system must reason from the image back toward properties of that world.

Chapter summary. Computer vision converts visual measurements into information about the world. It supports both geometric measurement and semantic interpretation, and it is closely related to computer graphics through the analysis–synthesis relationship.

CHAPTER 2

Why Vision Is Difficult

Vision is difficult because an image is an incomplete and ambiguous measurement. The world is three-dimensional, images are two-dimensional, and the appearance of the same object can vary dramatically with conditions.

2.1 An Ill-Posed Inverse Problem

The real world is much more complex than what can be measured in an image. Projection from 3D to 2D removes information, so image formation cannot simply be inverted. A vision system must infer hidden properties from evidence that is incomplete and often ambiguous.

2.2 Nuisance Parameters

Several factors change image appearance without necessarily changing the identity of the object or scene.

Table 2.1. Major sources of variation in visual appearance.

Factor	Effect on the observed image
Illumination	The same surface can appear different when lighting changes.
Object pose	Articulated or movable objects can change configuration.
Clutter	Irrelevant surrounding structure can interfere with interpretation.
Occlusion	Part of an object can be hidden by another object.
Intra-class appearance	Different members of the same category can look different.
Viewpoint	Appearance changes as the camera observes the object from a different direction.
Scale	The same object occupies different numbers of pixels at different distances or resolutions.
Deformation	Non-rigid objects may change shape.
Background clutter	Object evidence must be separated from a complicated background.

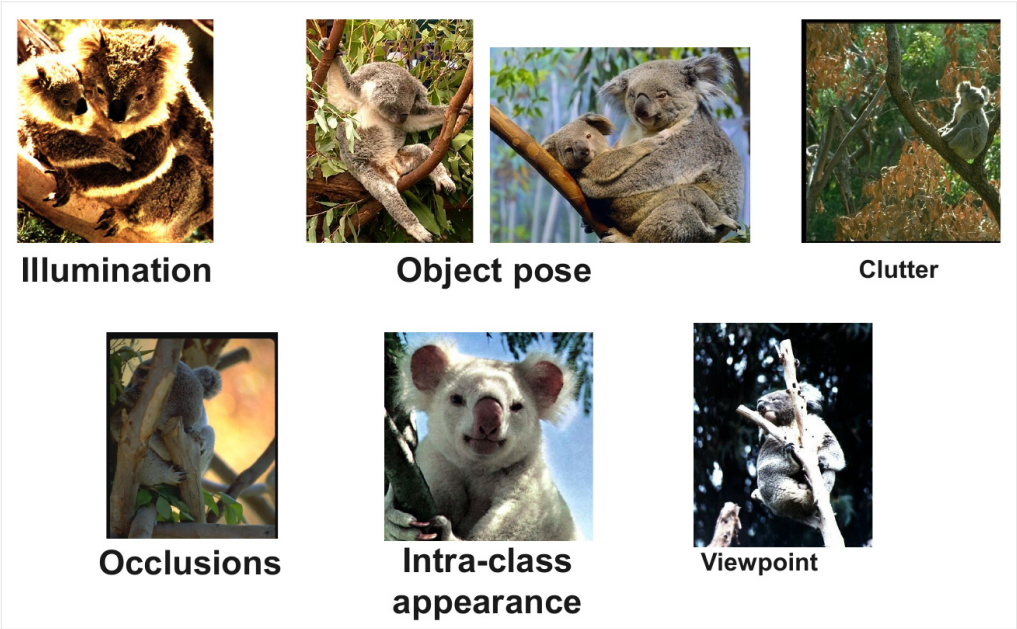


Figure 2.1. The same object category can change appearance because of illumination, pose, clutter, occlusion, intra-class differences, and viewpoint.

2.3 Context and Intra-Class Variation

Intra-class variation means that objects belonging to the same category may have very different visual appearances. Context is also important: a local region that is ambiguous in isolation can become easier to interpret when its surroundings are considered.

Viewpoint, illumination, scale, deformation, occlusion, and background clutter are therefore not exceptional cases. They are normal conditions that a useful vision system must tolerate.

2.4 Visual Cues

Images are difficult to interpret, but they also reveal structure through visual cues. The task is to exploit those cues rather than treating all variation as noise.

2.4.1 Depth and Ordering Cues

Linear perspective and aerial perspective provide information about depth. Occlusion provides depth ordering: when one object hides part of another, the occluding object is interpreted as being in front.

2.4.2 Shape and Lighting Cues

Texture gradients can support shape interpretation. Shading and cast shadows also provide information about shape, position, and illumination.

2.4.3 Grouping Cues

Similarity in color, texture, or proximity can group image elements into coherent structures. Common fate provides another grouping cue: image elements moving together are likely to belong together.

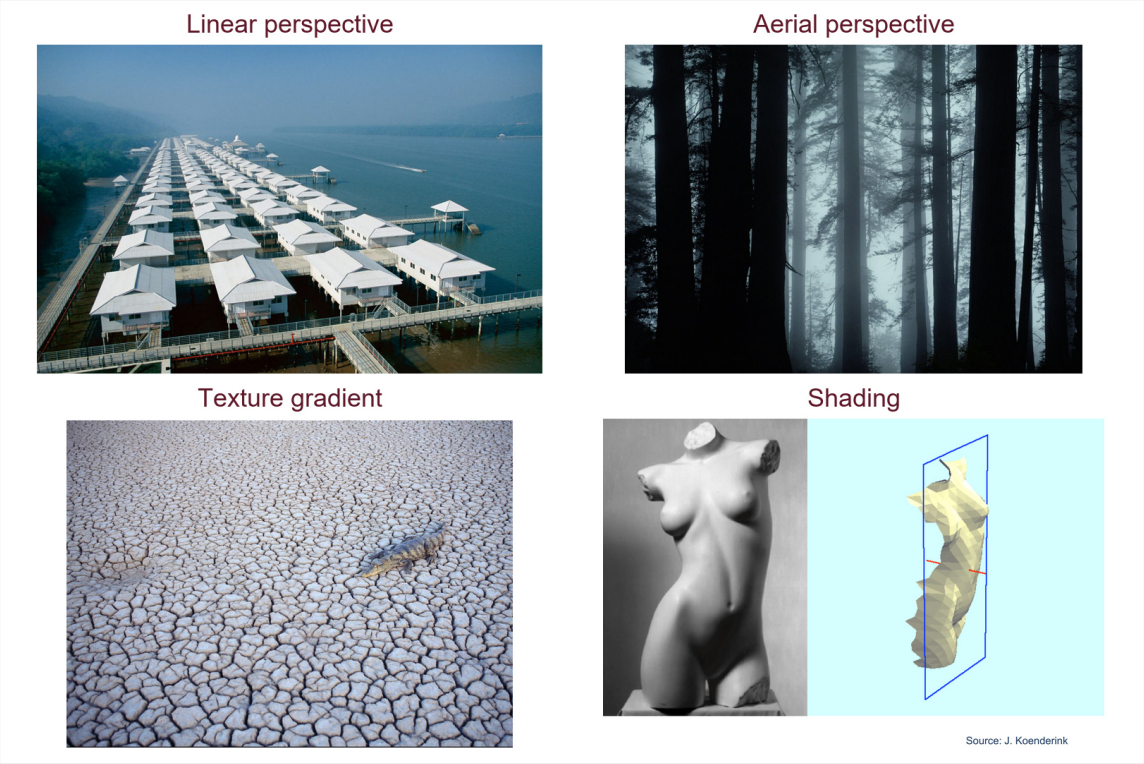


Figure 2.2. Examples of visual cues: linear perspective, aerial perspective, texture gradients, and shading.

2.5 Scale and Complexity

The computational scale of vision is large. A single image may contain thousands to millions of pixels. The space of human-recognizable object categories was described as roughly 3,000–30,000 categories, and articulated objects such as humans can involve more than 30 degrees of freedom. Visual data on the web add images, animated images, and video at very large scale.

A biological comparison reinforces the difficulty: approximately half of the cerebral cortex in primates has been associated with visual processing in the cited neurophysiological literature.

> **Chapter summary.** Vision is hard because the mapping from the world to an image is many-to-one and because appearance changes with nuisance parameters. Depth, shape, lighting, and grouping cues provide information that helps recover structure despite this ambiguity.

CHAPTER 3

Historical Development and Changing Emphasis

The history of computer vision shows a recurring shift between geometry, statistical modeling, data, and learning. Early optimism was followed by increasingly sophisticated representations and a recognition that visual understanding is not a short-term engineering problem.

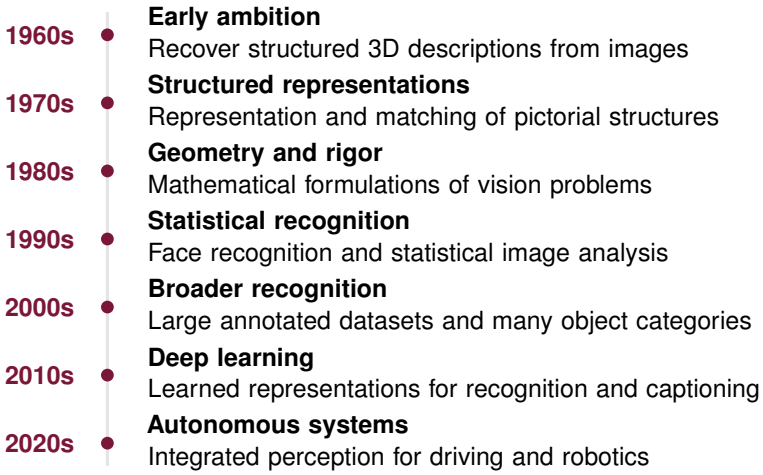


Figure 3.1. Source milestones reconstructed as a readable timeline; the following sections retain the historical examples.

3.1 The Early Ambition

In 1966, Marvin Minsky assigned a summer project whose goal was essentially to connect a camera to a computer and make the machine describe what it saw. Decades later, general visual understanding remained an open research problem.

Larry Roberts' 1963 MIT PhD work, *Machine Perception of Three-Dimensional Solids*, became an early landmark in interpreting synthetic scenes and recovering three-dimensional structure.

3.2 1970s: Structured Representations

Work in the 1970s made progress on selected images and structured visual representations. Fischler and Elschlager's 1973 work on the representation and matching of pictorial structures is a representative example.

3.3 1980s: Geometry and Mathematical Rigor

The 1980s saw artificial neural networks appear and then recede in influence while the field moved strongly toward geometry and more mathematically rigorous formulations.

3.4 1990s: Statistical Recognition

Face recognition became a prominent task, and statistical analysis became increasingly important in visual recognition.

3.5 2000s: Broader Recognition and Data

Recognition expanded to a broader set of categories. Large annotated datasets became increasingly important, and video processing grew as a major area of work.

3.6 2010s: Deep Learning

Deep learning returned to prominence. Representative milestones listed in the historical timeline include AlexNet (NIPS 2012), DeepFace (CVPR 2014), DeepPose (CVPR 2014), and *Show, Attend and Tell* (ICML 2015).

3.7 2020s: Autonomous Systems

Autonomous vehicles represent a major modern application direction in which recognition, geometry, motion, and real-time decision making must operate together.

> **Chapter summary.** Computer vision evolved from early synthetic-scene interpretation through structured representations, geometric rigor, statistical recognition, large annotated datasets, deep learning, and autonomous systems.

CHAPTER 4

Measurement and Semantic Understanding

Computer vision is useful both as a measurement technology and as a way to obtain semantic information. These two goals are complementary: one emphasizes geometry and physical structure, while the other emphasizes categories and meanings that matter to people.

4.1 Vision as a Measurement Device

Measurement-oriented vision includes real-time stereo, structure from motion, multi-view stereo, and three-dimensional reconstruction from collections of images. Space exploration provides a concrete example: visual systems can support terrain reconstruction, obstacle detection, and navigation.

> **Why learning still matters**
>
> For some physical measurements, other sensors can be more direct or accurate than cameras. However, visual navigation and semantic tasks still require learned or data-driven interpretation, because the goals of vision include both *what* is present and *where* it is.

4.2 Vision as a Source of Semantic Information

A street scene can support object labels, scene context, and qualitative spatial information.



Figure 4.1. Semantic interpretation can assign object and scene labels to visually meaningful regions.

Table 4.1. Examples of semantic information obtainable from one scene.

Level	Examples
Object categorization	sky, building, flag, face, banner, bus, wall, street lamp, cars
Scene / context	outdoor, city, traffic
Qualitative spatial information	horizontal, vertical, slanted, rigid moving object, non-rigid moving object

Semantic vision therefore includes scene type, spatial organization, and motion-related interpretation, not only isolated object labels. **Chapter summary.** Vision combines geometry and semantics: systems may need both *what* is present and *where* it is.

CHAPTER 5

Core Areas of Computer Vision

A useful organization of the field separates early vision, mid-level vision, multi-view geometry, recognition, and high-level or advanced topics. The categories are not completely independent, but they provide a practical map of the techniques studied in computer vision.

5.1 Early Vision: Image Formation and Processing

Early vision begins with image formation and basic image processing. Its topics include cameras, light, color, filtering, and edges.

5.1.1 The Physics of Imaging

Camera modeling addresses what a camera does and how camera position can be inferred. Light is studied in terms of measurement, interaction with surfaces, and the way image brightness is produced. Color is studied through its underlying mechanisms and through methods for describing and measuring it.

The broader topic list includes imaging geometry, pinhole cameras, lenses, camera parameters, and calibration.

5.1.2 Representing Small Image Patches

Small image neighborhoods are represented for three main reasons:

- to establish correspondence between points in different images;
- to detect sharp changes, commonly called edges;
- to represent texture through statistics of local image patterns.

Filter outputs can be interpreted as a dot-product-like comparison between a pattern and the image while the pattern is shifted across image locations. A strong response indicates that the local image resembles the pattern. Derivative-like filters are useful for detecting intensity changes.

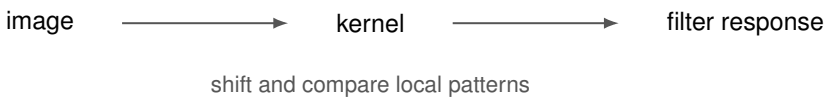


Figure 5.1. Conceptual filtering pipeline: a kernel is shifted across the image to produce local responses.

Image filtering and enhancement also include linear filters, convolution, smoothing, edge detection, and image pyramids.

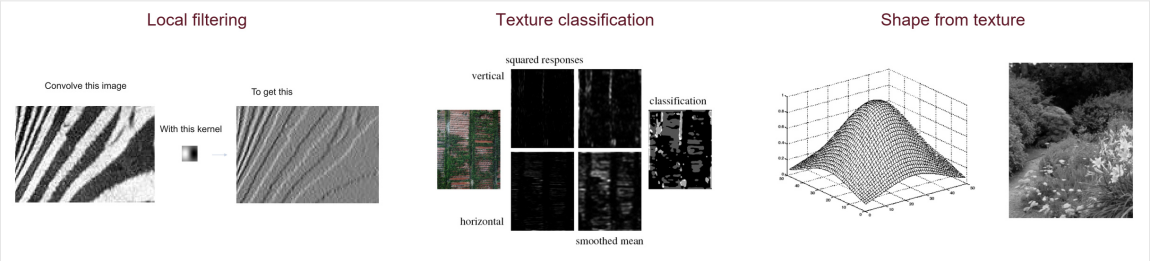


Figure 5.2. Early-vision examples: local filtering, texture classification from filter responses, and shape information derived from texture.

5.1.3 Texture

Many objects are distinguished by texture. Tigers, cheetahs, grass, and trees provide intuitive examples. Texture can be represented through statistics of filter outputs: coarse bar filters respond strongly to tiger-like stripes, spot filters to cheetah-like patterns, long narrow filters to grass-like structure, and extended spot-like responses to leaves.

Objects with different textures can be segmented, and variation in texture can provide a cue to three-dimensional shape.

5.2 Mid-Level Vision: Grouping, Fitting, and Tracking

Mid-level vision seeks coherent structure by grouping or fitting lower-level measurements. Fitting can include alignment and least-squares methods.

5.2.1 Segmentation

Segmentation breaks an image or video into useful pieces. Examples include finding video sequences corresponding to one shot and finding image components that are coherent in internal appearance.

5.2.2 Tracking

Tracking keeps track of a moving object through a long sequence of views.

5.3 Multi-View Geometry

Multi-view geometry studies relationships across several views. The core examples are stereo and projective structure from motion.

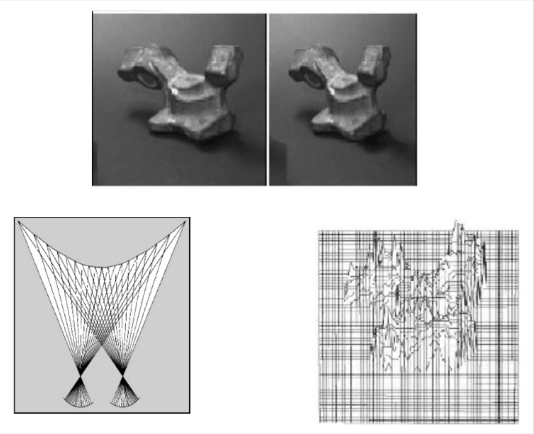


Figure 5.3. Stereo vision uses multiple views to recover geometric information that is not directly available from a single image.

5.4 Recognition

A representative recognition pipeline consists of patch description and matching, clustering and visual vocabularies, bag-of-features models, and classification.



Figure 5.4. Recognition pipeline from local patch description to classification.

5.5 High-Level Geometric Vision

High-level geometric vision studies relations between object geometry and image geometry. Its topics include model-based vision for finding the position and orientation of known objects, smooth surfaces and outlines, aspect graphs that describe how outlines change with viewpoint, and range data.

5.6 Advanced Topics

Representative advanced topics include segmentation, face detection, and articulated models. The broader topic list also includes image restoration, shape analysis, motion and optical flow, semantic detection, and image mosaics.

5.7 Image Collections and Image-Based Rendering

Two recurring application directions are finding images in large collections and image-based rendering. Image collections support searching and browsing. Image-based rendering addresses the difficulty of building geometric models that reproduce every real-world detail, such as surface weathering, by producing new pictures from existing images.

> **Chapter summary.** Early vision studies image formation and local measurements. Mid-level vision groups and fits those measurements. Multi-view geometry reasons across viewpoints, recognition assigns categories, and high-level topics connect image evidence to object geometry and complex applications.

CHAPTER 6

Applications of Computer Vision

Computer vision appears in systems ranging from document analysis to robotics and medical imaging. The examples below illustrate how the same core ideas recur in different practical settings.

6.1 Detection and Recognition

A basic detection task asks whether an image contains a person. The positive and negative examples used for this problem emphasize the need to distinguish target instances from images without the target.



Figure 6.1. Positive and negative examples illustrate the basic structure of a visual detection problem.

Face detection in digital cameras is a successful example: detected faces can be used to control autofocus, exposure, and flash output.

Smile detection and face recognition are further examples in which visual information is used to infer a specific facial state or identity.

6.2 Optical Character Recognition

Optical character recognition converts scanned documents into text. Related examples include handwritten digit recognition and automatic license-plate reading.

6.3 Retail and Mobile Recognition

Retail vision can detect and recognize products. A historical example is LaneHawk, in which a camera mounted at a checkout lane detects items under a shopping basket so that the cashier can verify the detected quantity.

Mobile-phone recognition extends the same idea to camera-equipped phones. Examples included Microsoft Research systems and Nokia Point & Find.

6.4 Biometrics and Password-Free Login

Vision-based biometrics includes face, iris, and fingerprint recognition. The identification of the "Afghan Girl" from iris patterns is an illustrative example of iris recognition. Fingerprint scanners and face-recognition systems also support login without a conventional password.

6.5 Special Effects and Interaction

Shape capture and motion capture are used in visual effects. Examples include work associated with *The Matrix* and *Pirates of the Caribbean*.

Vision-based interaction includes camera-based tracking for games and interfaces. Examples include infrared tracking with the Nintendo Wii and camera tracking of crowds for interactive displays.

6.6 Vehicles, Robotics, and Space Exploration

Applications include autonomous cars, planes, missiles, robots, space exploration, and manufacturing or quality control. Smart-car systems use vision for road-scene interpretation.

A historical automotive example cited Mobileye systems in high-end BMW, GM, and Volvo models and projected broad manufacturer adoption by 2010.

Mars exploration provides a detailed robotics example. Vision systems at JPL were used for panorama stitching, three-dimensional terrain modeling, obstacle detection, and position tracking. Robotics examples also include the Mars Spirit Rover and RoboCup.

6.7 Medical and Scientific Imaging

Medical imaging includes three-dimensional imaging with modalities such as MRI and CT, as well as image-guided surgery.

6.8 Image Retrieval and Three-Dimensional Viewing

Image retrieval supports finding and browsing pictures in large collections. Three-dimensional Earth viewers are examples of large-scale three-dimensional modeling and visualization.

6.9 Application Overview

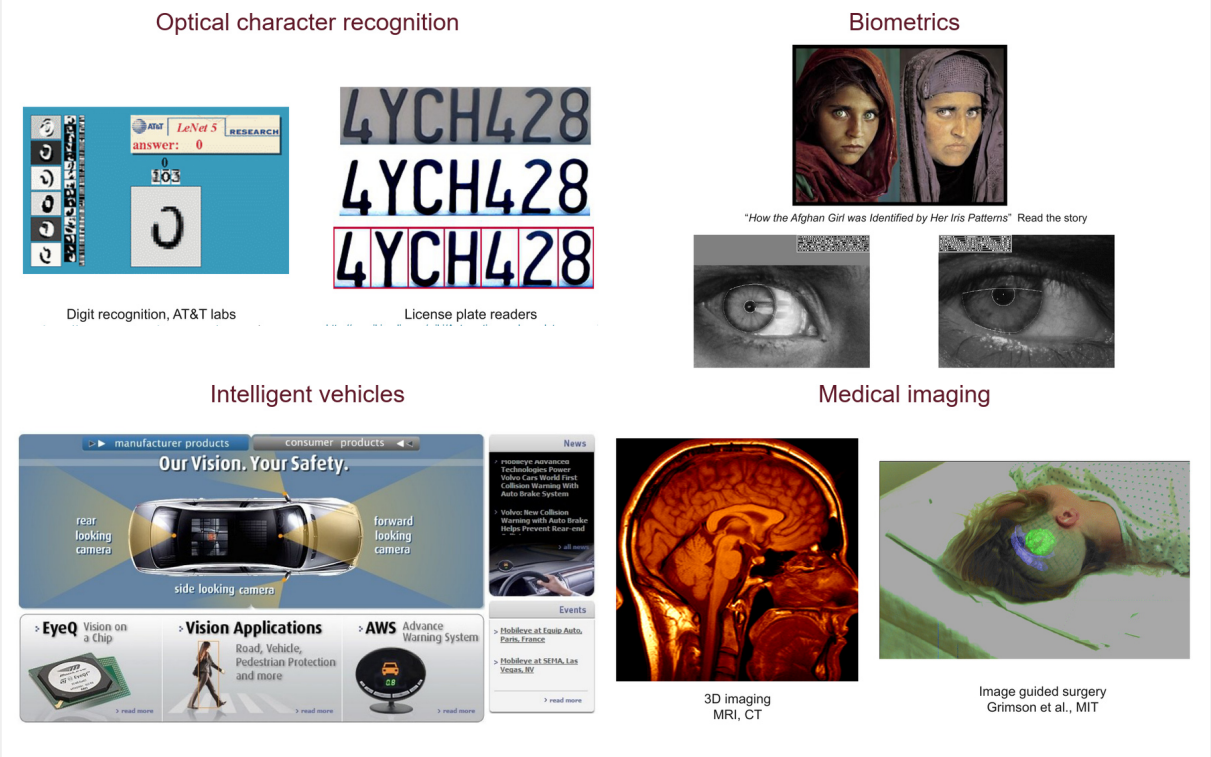


Figure 6.2. Examples of computer-vision applications in OCR, biometrics, intelligent vehicles, and medical imaging.

Table 6.1. Representative computer-vision application areas.

Application area	Representative examples
Transportation and autonomy	smart cars, autonomous vehicles, planes, robots
Space exploration	panorama stitching, 3D terrain modeling, obstacle detection, tracking
Accessibility	aid to the blind, ASL recognition
Manufacturing	quality control, retail product recognition
Security and identity	surveillance, biometrics, face / iris / fingerprint recognition
Documents	OCR, digit recognition, license-plate reading
Medical imaging	MRI, CT, 3D imaging, image-guided surgery
Entertainment and interaction	special effects, motion capture, camera-based interaction
Image collections	retrieval, browsing, image-based rendering

Chapter summary. Computer vision supports recognition, documents, retail, biometrics, special effects, interaction, autonomous systems, robotics, space exploration, medical imaging, and image retrieval. These applications combine different subsets of sensing, geometry, grouping, motion, and recognition.

CHAPTER 7

Computer Vision and Machine Learning

Machine learning is an important tool for computer vision, but the two fields are not identical. Computer vision also includes sensing, image formation, geometric modeling, and visual representation.

7.1 The Role of Machine Learning

Machine learning studies algorithms that can change behavior based on data. A major goal is to learn complex patterns and make decisions from data. This is directly useful in computer vision, especially for recognition.

Computer vision, however, is more than learning. It also includes explicit modeling and example-based learning, and it cares about how visual data are obtained and represented.

Table 7.1. Contrast between the emphases described for machine learning and computer vision.

	Machine learning emphasis	Computer vision emphasis
Data	Learns patterns and decisions from data	Also considers how visual data are acquired
Sensors	Often treats the data source as given	Includes sensor design and active vision
Representation	Learns or uses representations for prediction	Explicitly studies visual representation
Modeling	Data-driven methods are central	Modeling and learning both matter

The source's pattern-recognition example separates **training** from **testing**. Selected measurements (features) from training images are used to train a statistical method; corresponding measurements from testing images are then used to predict a category. Representative methods shown are support vector machines (SVM), Gaussian Naive Bayes (GNB), and neural networks.



Figure 7.1. Training and testing in the lecture's statistical pattern-recognition example.

7.2 Related Disciplines

Computer vision is connected to image processing, pattern recognition, photogrammetry, computer graphics, artificial intelligence, machine learning, projective geometry, and control theory.

7.3 Active Research Topics

Representative research topics include object recognition, human behavior analysis, continuous computer-vision streams, biometrics and soft biometrics, large-scale three-dimensional reconstruction at city scale, medical image processing, and vision for robotics.

7.4 Research Publications

Major publication venues include *IEEE Transactions on Pattern Analysis and Machine Intelligence* (TPAMI), the *International Journal of Computer Vision* (IJCV), and *IEEE Transactions on Image Processing*. Major conferences include ICCV, CVPR, and ECCV.

> **Historical publication snapshot**
>
> A historical publication snapshot lists TPAMI with an impact factor of 17.730 and IJCV with 11.541. These values are historical rather than current metrics. CVPR is described as annual, while ICCV and ECCV are described as occurring every two years.

The publication rankings are used to emphasize the importance of computer vision research within computer science and electrical engineering.

7.5 Further Reading

Recommended and optional books include:

- David Forsyth and Jean Ponce, *Computer Vision: A Modern Approach*;
- Richard Szeliski, *Computer Vision: Algorithms and Applications*;
- Emanuele Trucco and Alessandro Verri, *Introductory Techniques for 3-D Computer Vision*;
- Rafael C. Gonzalez and Richard E. Woods, *Digital Image Processing*.

> **Chapter summary.** Machine learning provides important methods for visual recognition, but computer vision additionally studies cameras, sensors, image formation, geometry, representation, and visual data acquisition. The field connects to several neighboring disciplines and remains active across recognition, reconstruction, biometrics, medicine, and robotics.

Lecture 1 Quick Review

Foundations and visual cues

- **Purpose**: extract geometric, spatial, and semantic properties of the 3D world from images; graphics follows the synthesis direction.
- **Ambiguity**: 3D-to-2D projection is ill-posed. Illumination, pose, occlusion, viewpoint, scale, deformation, clutter, and intra-class variation affect appearance; context helps interpretation.
- **Depth and shape**: linear/aerial perspective, occlusion, texture gradients, shading, and cast shadows provide different cues.
- **Grouping and scale**: similarity and common fate help organization; millions of pixels and many categories make vision computationally demanding.

History and understanding

- Development moved through structured representations, geometry, statistics, annotated datasets, and deep learning.
- **Measurement** includes stereo, structure from motion, and multi-view reconstruction; **semantics** includes objects, scenes, and qualitative spatial relations.

Core areas and methods

- **Early vision**: image formation, cameras, light, color, filtering, edges, and local patches. Shifted filters compare image patches with patterns.
- **Texture**: filter-response statistics support segmentation and shape interpretation; patches also support correspondence.
- **Mid-level and multi-view**: fitting, grouping, segmentation, tracking, stereo, and projective structure from motion.
- **Recognition**: patch descriptions, visual vocabularies, bag-of-features, and classification. Higher geometric vision studies models, surfaces, outlines, aspect graphs, and range data.

Applications and learning

- OCR, biometrics, medicine, autonomous systems, robotics, space, effects, interaction, and retrieval combine sensing, geometry, and recognition.
- Face detection can control autofocus, exposure, and flash. Rover vision supports panoramas, terrain models, obstacles, and position tracking.
- **Machine learning** learns patterns from data; vision additionally studies sensing, acquisition, modeling, and representation.
- Related fields include image processing, pattern recognition, photogrammetry, graphics, AI, geometry, learning, and control.
- Research includes recognition, behavior, biometrics, reconstruction, medicine, and robotics. Major venues include TPAMI, IJCV, IEEE TIP, CVPR, ICCV, and ECCV.

LECTURE 2

Image Acquisition and Digitalization

From scene illumination to a sampled digital image.

Image formation · Digitalization · Pixel relations · Interpolation

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 8

Image Formation and Camera Acquisition

The acquisition lecture follows the physical path from illumination to a digital image. A scene point receives incident light, the surface modifies that light according to its reflectance and other optical properties, the reflected light is projected onto the image plane, and the sensor converts wavelength-dependent energy into one or more measured image channels.

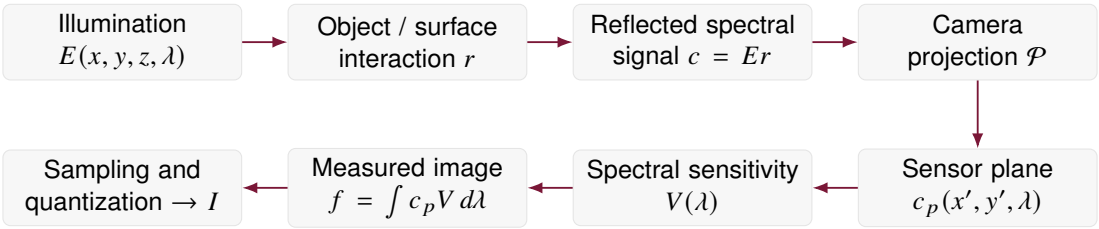


Figure 8.1. Acquisition chain connecting the lecture's physical, geometric, spectral, and digital stages. Symbols are defined in the following equations; I is the digital image.

8.1 Illumination and Surface Properties

The slides contrast typical source strengths and surface reflectivities to emphasize that observed brightness depends on both illumination and the object. Representative values shown in the lecture are collected in Table 8.1.

Table 8.1. Representative illumination and reflectivity values reported in the lecture.

Light source	i (candela)	Object / surface	r
Solar light	9000	Black velvet	0.01
Cloudy sky	1000	White wall	0.80
Moon light	0.01	Silver and other metals	0.90
Internal office light	100	Snow	0.93

The lecture distinguishes five surface interactions: **absorption**, **diffusion**, **reflection**, **transparency**, and **refraction**. They describe different ways in which incoming light can be attenuated, scattered, redirected, transmitted, or bent at an interface. The wavelength λ matters because both illumination and material response can vary spectrally.

8.2 A Spectral Image-Formation Model

For a natural image, let $E(x, y, z, \lambda)$ denote the incident light at a world point and let $r(x, y, z, \lambda)$ denote the point's reflectivity. The reflected light is modeled in the lecture as

$$c(x, y, z, \lambda) = E(x, y, z, \lambda)\, r(x, y, z, \lambda). \tag{8.1}$$

where

- $E(x, y, z, \lambda)$: incident spectral light at world location (x, y, z);
- $r(x, y, z, \lambda)$: wavelength-dependent surface reflectivity;
- $c(x, y, z, \lambda)$: reflected spectral light leaving that point toward the imaging system;
- λ: wavelength.

> **From scene radiance to an image**
>
> The camera does not directly store the three-dimensional spectral function. It first projects world coordinates into two-dimensional image coordinates and then applies the wavelength sensitivity of the capture device.

The projection operator $\mathcal{P}$ maps a world point to camera coordinates:

$$c_p(x', y', \lambda) = \mathcal{P}(c(x, y, z, \lambda)) \, . \tag{8.2}$$

where

- (x, y, z): world coordinates;
- (x', y'): image-plane or camera coordinates;
- c_p: projected spectral signal at the sensor plane;
- $\mathcal{P}$: camera projection operator.

Once the projected signal reaches the sensor, the device sensitivity $V(\lambda)$ determines how much of each wavelength contributes to the measured image value:

$$f(x', y') = \int c_p(x', y', \lambda) V(\lambda) \, d\lambda. \tag{8.3}$$

where

- $V(\lambda)$: spectral sensitivity function of the capture device;
- $f(x', y')$: scalar image function measured at image location (x', y');
- the integral: accumulation of wavelength-dependent energy weighted by sensor sensitivity.

8.3 Perspective and Orthographic Projection

The lecture compares two projection models. In a **perspective projection**, objects closer to the camera appear larger; this matches the human eye and ordinary cameras and is therefore natural, but mathematically more involved. In an **orthographic projection**, apparent object size is independent of distance; this is less realistic but simpler to analyze.

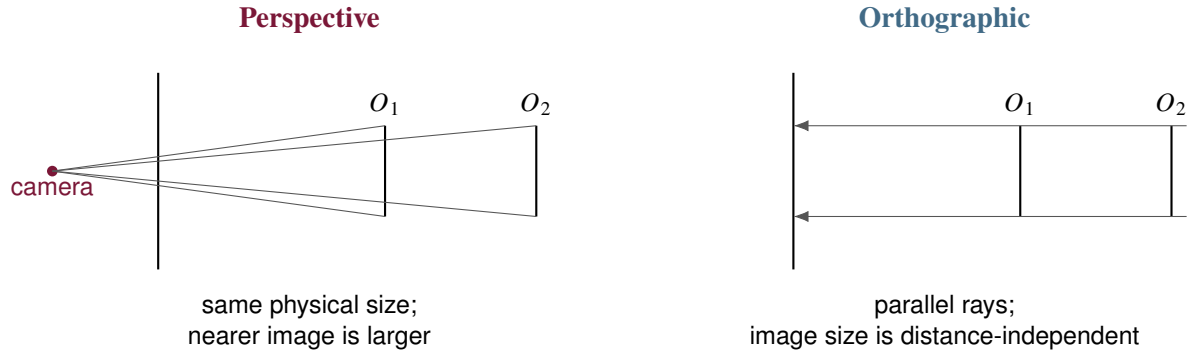


Figure 8.2. Conceptual contrast between perspective and orthographic projection.

8.4 Color Sensors

A color camera can be modeled as three wavelength-selective measurements at each image location. With red, green, and blue sensitivity functions V_R, V_G, and V_B, the lecture writes

$$\begin{aligned} f_R(x', y') &= \int c_p(x', y', \lambda) V_R(\lambda)\, d\lambda, \\ f_G(x', y') &= \int c_p(x', y', \lambda) V_G(\lambda)\, d\lambda, \\ f_B(x', y') &= \int c_p(x', y', \lambda) V_B(\lambda)\, d\lambda. \end{aligned} \tag{8.4}$$

where

- f_R, f_G, f_B: the three measured image channels;
- V_R, V_G, V_B: red-, green-, and blue-tuned sensor sensitivity functions;
- c_p: projected spectral signal reaching the sensor.

The three channel values are then sufficient for ordinary RGB display devices to reconstruct a color appearance, even though the original spectrum contains much more information.

> **Chapter summary.** Image formation is a chain: illumination → surface interaction → reflected spectral light → geometric projection → sensor sensitivity. Perspective and orthographic projection differ in whether apparent size changes with depth, while color acquisition uses multiple spectral sensitivity functions rather than one scalar response.

CHAPTER 9

Digital Images: Sampling, Quantization, and Resolution

9.1 From a Continuous Image to a Digital Image

Before digitalization, the projected image is continuous in both its spatial domain and its value range: $(x', y') \in \mathbb{R}^2$ and $f(x', y') \in \mathbb{R}$. A digital computer therefore needs two discretization steps.

> **Sampling and quantization**
>
> **Sampling** discretizes spatial coordinates, producing a finite grid of locations. **Quantization** discretizes the value measured at each sampled location, producing one of a finite set of representable levels.

The lecture expresses the two operations schematically as

$$(x', y') \longrightarrow (x'_i, y'_j), \qquad i = 0, \ldots, N-1, \quad j = 0, \ldots, M-1, \tag{9.1}$$

$$f(x'_i, y'_j) \longrightarrow \hat{f}(x'_i, y'_j) \in \mathbb{N}. \tag{9.2}$$

where

- N, M: numbers of sample positions along the two image axes;
- (x'_i, y'_j): discrete sample location;
- f: continuous-valued image before quantization;
- $\hat{f}$: quantized digital value.

The sampling slides represent spatial sampling as multiplication by a finite **comb function**:

$$\text{comb}(x', y') = \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} \delta(x' - i\Delta_x,\ y' - j\Delta_y). \tag{9.3}$$

where

- δ: two-dimensional Dirac impulse selecting a grid location;
- Δ_x, Δ_y: spacing between samples along the two axes;
- i, j: grid indices, with N horizontal and M vertical sample positions;
- x', y': continuous image-plane coordinates.

The slide writes the operation schematically as

$$f(x'_i, y'_j) = f(x', y') \times \text{comb}(x', y'), \qquad f(x'_i, y'_j) = f(i, j) \text{ for simplicity.} \tag{9.4}$$

Here $x'_i = i\Delta_x$ and $y'_j = j\Delta_y$; $f(i, j)$ denotes the value at that sample, before quantization. The product on the right is an impulse representation of the sampled image; its impulse weights are the sample values on the left. This distinction makes the slide's shorthand precise without changing its sampling construction.

A digital image is therefore a multidimensional array. For a monochrome image, a pixel stores one intensity value; for a color image, it stores a vector of channel values. The lecture also notes other coordinate structures: (x, y, z) for volumetric images such as CT scans and (x, y, t) for video.

9.2 Quantization and Bit Depth

If each pixel value is represented by b bits, the number of available quantization levels is

$$N_c = 2^b. \tag{9.5}$$

where

- N_c: number of representable gray levels or colors;
- b: number of bits used for each quantized value.

For example, $b = 8$ gives $2^8 = 256$ levels. Reducing the number of levels creates visible **false contours** because smooth intensity changes collapse into coarse steps.

After sampling and quantization, the lecture describes an image as a discrete function with a finite discrete domain and range:

$$\Omega \subset \mathbb{N}^2, \qquad I : \Omega \longrightarrow \{1, \ldots, P\}, \qquad P \in \mathbb{N}. \tag{9.6}$$

where

- Ω: finite set of pixel coordinates selected by sampling;
- I: image assigning a quantized value to each pixel;
- P: number of representable levels; the source's 8-bit example uses $P = 2^8 = 256$ and $\log_2 P = 8$.

The slide numbers the levels from 1 to P; this expresses the number of levels, while binary image values elsewhere use 0 and 1.

Digital image processing means processing digital, multidimensional images using a digital computer.

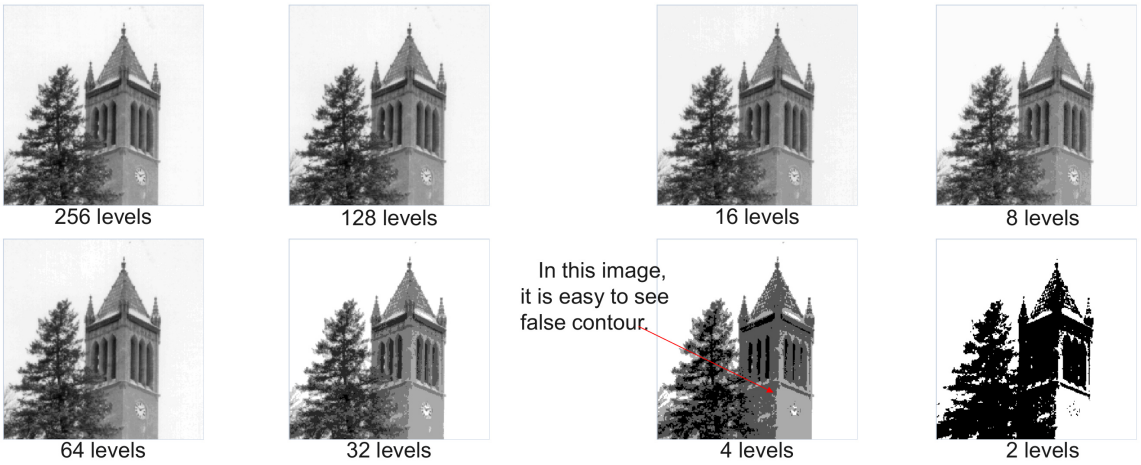


Figure 9.1. Source examples of progressively coarser quantization; false contouring becomes evident at very low bit depth.

9.3 Digital Image Types

Table 9.1. Digital image representations presented in the lecture.

Type	Pixel content	Interpretation
Intensity / monochrome	one scalar	light intensity, commonly displayed as gray level
RGB color	a 3-component vector	red, green, and blue channel values
Binary	one bit	0 for black and 1 for white in the lecture convention
Indexed	one integer index	lookup into a color table containing RGB triples

9.4 Pixels, Memory, and Spatial Resolution

A **pixel** is a single value on the sampled grid. The lecture stresses that a pixel should be understood as the smallest treatable image area, not as an infinitesimal geometric point.

For an $M \times N$ image with L intensity levels, binary storage uses

$$k = \lceil \log_2 L \rceil, \qquad B = MNk, \tag{9.7}$$

where

- M, N: image dimensions in pixels;
- L: number of intensity levels;
- k: bits per pixel value;
- B: total number of bits for the image.

> **Notation in the source**
>
> One slide writes $k = \log(L)$, while the later 8-bit example explicitly uses $\log_2(P) = 8$ for $P = 256$. The binary-storage interpretation therefore uses base-2 logarithms, rounded up when L is not a power

of two. The formula counts uncompressed, fixed-length monochrome pixel data; multiply by the number of channels when each channel uses k bits.

Spatial resolution is the smallest distinguishable detail in an image. The lecture lists pixels per inch (ppi), dots per inch (dpi), and raw $M \times N$ dimensions as common descriptors, while warning that pixel dimensions alone do not fully express the relation between digital sampling and the detail present in the original scene.

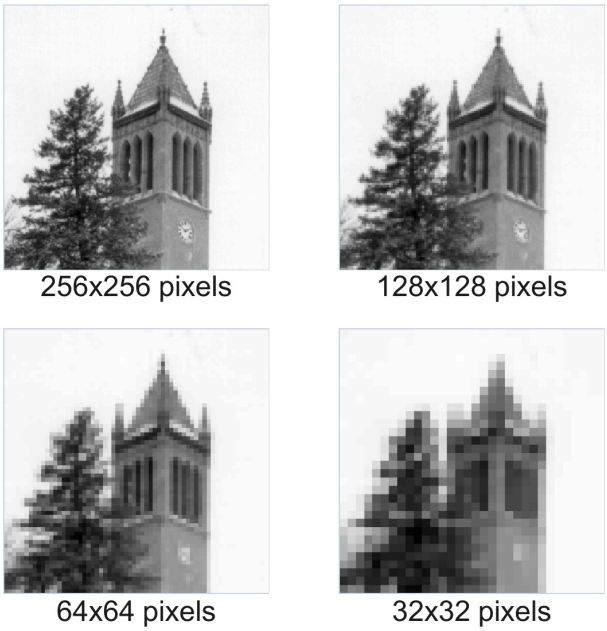


Figure 9.2. Effect of reducing spatial sampling while preserving the same field of view: fine detail is progressively lost.

Under-sampling is irreversible: interpolation can produce a denser grid, but it cannot recover scene details that were never sampled. The lecture gives application-dependent resolution examples, such as monitor/web display around 72 dpi and printing at substantially higher dpi, and presents interpolation as an approximation when increasing image size.

Resolution and perceived quality

For images displayed at the same physical size, insufficient spatial sampling produces visible pixels and lost detail. If the pixel dimensions are kept fixed instead, changing the number of samples also changes the displayed image size. The suitable combination of image size and pixel depth is application- and content-dependent.

Chapter summary. Digitalization separates two independent limitations: spatial sampling controls where the image is measured, while quantization controls how finely each measured value is represented. Bit depth determines the number of levels; spatial resolution determines recoverable detail; interpolation only estimates values between existing samples.

CHAPTER 10

Pixel Relationships and Image Resampling

10.1 Neighborhoods

For a pixel $p = (x, y)$, the lecture defines the four-neighborhood, diagonal neighborhood, and eight-neighborhood:

$$N_4(p) = \{(x-1, y), (x+1, y), (x, y-1), (x, y+1)\}, \tag{10.1}$$

$$N_D(p) = \{(x-1, y-1), (x+1, y-1), (x-1, y+1), (x+1, y+1)\}, \tag{10.2}$$

$$N_8(p) = N_4(p) \cup N_D(p). \tag{10.3}$$

where

- $p = (x, y)$: reference pixel;
- $N_4(p)$: horizontal and vertical neighbors;
- $N_D(p)$: four diagonal neighbors;
- $N_8(p)$: all eight surrounding grid locations.

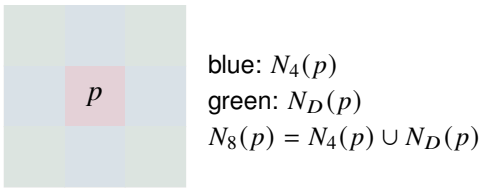


Figure 10.1. Pixel-neighborhood relations used to define connectivity, adjacency, and paths.

10.2 Connectivity, Adjacency, and Paths

Two pixels from the same class are connected when the selected neighborhood relation links them. The lecture distinguishes **4-connectivity**, **8-connectivity**, and **mixed (m-) connectivity**. Let V be the set of pixel values defining the selected class. For pixels p, q with values in V, m-adjacency holds if either $q \in N_4(p)$, or $q \in N_D(p)$ and no pixel in $N_4(p) \cap N_4(q)$ has a value in V. Connectivity means that a path of adjacent pixels exists; the adjacency rule itself describes a single step. The mixed rule is introduced to avoid the ambiguity that diagonal links can create under pure 8-connectivity.

> **Restrict the rule to the selected class**
>
> The source's written rule omits the restriction to the selected pixel class. Diagonal pixels on an interior grid always share two 4-neighbors; m-adjacency requires that neither shared neighbor belongs to the selected class. In this diagram, the dashed diagonal is excluded because it shares a selected neighbor, whereas the final diagonal to q is allowed because its shared neighbors are outside the class.

A pixel p is **adjacent** to q when the selected local neighborhood rule links them. Two subsets S_1 and S_2 are adjacent if some pixel in one subset is adjacent to a pixel in the other. A path from $p = (x, y)$ to

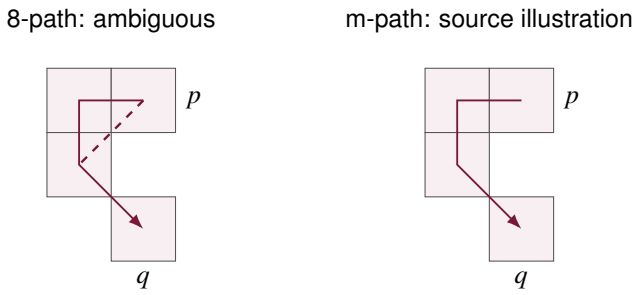


Figure 10.2. The source example removes the alternative diagonal route in its m-path illustration, retaining a single route from p to q.

$q = (s, t)$ is a sequence of distinct adjacent pixels $(x_0, y_0), \ldots, (x_n, y_n)$ with the required endpoints; the adjacency convention determines whether the result is a 4-path, 8-path, or m-path.

10.3 Distance Measures

A distance D must satisfy non-negativity, identity of indiscernibles, symmetry, and the triangle inequality. Three distances appear in the lecture:

$$D_E(p, q) = \sqrt{(x - s)^2 + (y - t)^2}, \tag{10.4}$$

$$D_4(p, q) = |x - s| + |y - t|, \tag{10.5}$$

$$D_8(p, q) = \max\{|x - s|, |y - t|\}. \tag{10.6}$$

where

- $p = (x, y)$ and $q = (s, t)$: two pixel coordinates;
- D_E: Euclidean distance;
- D_4: city-block distance, whose unit-distance pixels are exactly the 4-neighbors;
- D_8: chessboard distance, whose unit-distance pixels are exactly the 8-neighbors.

10.4 Interpolation

Interpolation estimates the value of an image at non-integer coordinates. The lecture separates resizing into two steps:

1. **Create the new sampling grid**, mapping its positions into the original image coordinates. The slide's $1.5\times$ zoom changes a 500×500 grid to 750×750.
2. **Assign a value to each new pixel** by interpolation of the existing samples.

The lecture compares four approaches.

Nearest-neighbor interpolation is written in the slides as

$$f(x, y, z) = \mathrm{Im}(\mathrm{round}(x), \mathrm{round}(y), z), \tag{10.7}$$

Here $f(x, y, z)$ is the interpolated image value, Im is the original sampled image, x, y are source spatial coordinates, round selects the nearest integer, and z remains an integer channel index in the implementation example.

For triangle interpolation, the slide uses the schematic weighted sum

$$Q = V_1 A_1 + V_2 A_2 + V_3 A_3, \tag{10.8}$$

Table 10.1. Interpolation methods discussed in the acquisition lecture.

Method	Principle	Lecture emphasis
Nearest neighbor	choose the closest sampled pixel	fastest; blocky appearance; be careful with integer rounding
Triangle interpolation	weighted combination of three triangle vertices	useful for less-structured samples; weights follow opposite sub-areas and should be normalized
Bilinear	weighted combination of the four corners of a grid cell	smoother than nearest neighbor; four lookups; often a good speed/quality tradeoff
Bicubic	cubic interpolation using a 4×4 neighborhood	16 neighbors; smoother than bilinear; more computation

Here Q is the value assigned at the interior point q, V_i are vertex values, and A_i are opposite sub-triangle areas, for $i = 1, 2, 3$. This is the **slide's schematic expression**. The slide explicitly requires normalization by total area in the general case. Its **normalized interpretation** is

$$Q = \frac{V_1A_1 + V_2A_2 + V_3A_3}{A_1 + A_2 + A_3}. \tag{10.9}$$

The denominator is the whole triangle's area; the weights $A_i/(A_1 + A_2 + A_3)$ therefore sum to one.

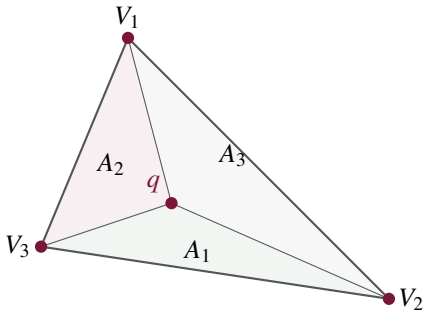


Figure 10.3. Triangle construction: A_i is the sub-triangle opposite vertex V_i. Normalize by $A_1 + A_2 + A_3$ to obtain area weights.

For bilinear interpolation inside a unit grid cell, the four opposite rectangle areas are

$$A_1 = d_2d_4, \quad A_2 = d_1d_4, \quad A_3 = d_2d_3, \quad A_4 = d_1d_3, \tag{10.10}$$

and the interpolated value is

$$Q = V_1A_1 + V_2A_2 + V_3A_3 + V_4A_4. \tag{10.11}$$

where

- Q: interpolated value at the position q; $V_1, \ldots, V_4$: values at the four surrounding grid points;
- d_1, d_2, d_3, d_4: horizontal and vertical fractional distances shown in the lecture construction;
- A_i: area opposite vertex V_i; within a unit cell the weights sum to one.

Bicubic interpolation is presented as a cubic interpolation of four cubic interpolations. It uses the 16 nearest samples, fits third-order polynomials such as $f(x) = a + bx + cx^2 + dx^3$, interpolates along one axis, and then fits/interpolates along the other. Here x is the interpolation coordinate, $f(x)$ is its estimated value, and a, b, c, d are the fitted polynomial coefficients.

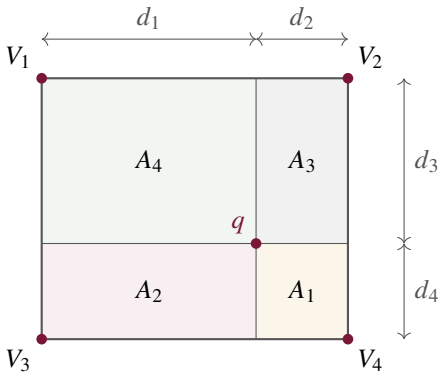


Figure 10.4. Bilinear geometry redrawn from the source: $d_1 + d_2 = d_3 + d_4 = 1$ in the unit cell. Each corner uses the opposite rectangle's area; Q is the value at q. The drawing is enlarged for readability.



Figure 10.5. Lecture comparison of bilinear and bicubic resampling on the same image region.

The resize algorithm is consequently simple at a high level: for every pixel in the new image, map its position back into the original-image coordinates, interpolate the source value, and write that value into the output image.

> **Chapter summary.** Neighborhood relations define connectivity, adjacency, paths, and grid-aware distance measures. Resampling then estimates values between grid points: nearest neighbor is cheapest, bilinear provides a common compromise, and bicubic trades additional computation for smoother results. Increasing sample density by interpolation does not reverse information lost during down-sampling.

Lecture 2 Quick Review

Physical acquisition

1. Reflected light is modeled by incident illumination multiplied by surface reflectivity.
2. Projection maps the 3D scene into 2D image coordinates.
3. Sensor sensitivity integrates the projected spectrum over wavelength.
4. RGB acquisition uses three different spectral sensitivity functions.
5. Perspective projection changes apparent size with depth; orthographic projection does not.

Digitalization and storage

6. Sampling discretizes position; quantization discretizes value.
7. b bits provide 2^b representable levels.
8. Low quantization depth can create false contours.
9. A pixel is the smallest treatable sampled image area.
10. For $M \times N$ pixels and k bits per value, storage is MNk bits.

Pixel relations and interpolation

11. N_4, N_D, and N_8 define local grid neighborhoods.
12. m-connectivity avoids some diagonal ambiguity of 8-connectivity.
13. D_4 is city-block distance; D_8 is chessboard distance.
14. Nearest neighbor is fast but blocky; bilinear uses four samples; bicubic uses sixteen.
15. Interpolation creates estimates, not lost high-frequency detail.

LECTURE 3

Color, Perception, and Texture

From spectral measurements to color and appearance.

Human vision · Color spaces · White balance · Texture

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 11

Color: Light and Human Vision

11.1 The Nature of Color

The lecture emphasizes that color is not simply a physical property attached to an object. It is a **psychological property of visual experience** produced by the interaction between the spectrum of physical light and the human visual system. This distinction motivates both colorimetry and color constancy: a camera measures spectral effects, whereas a human observer experiences a perceptual color.

Visible light occupies only a limited interval of the electromagnetic spectrum, approximately 400–700 nm in the lecture. A light source can be described physically by its spectral power distribution: the amount of energy emitted at each wavelength. The lecture contrasts narrow-band sources such as a ruby laser with broader spectra such as tungsten illumination and daylight.

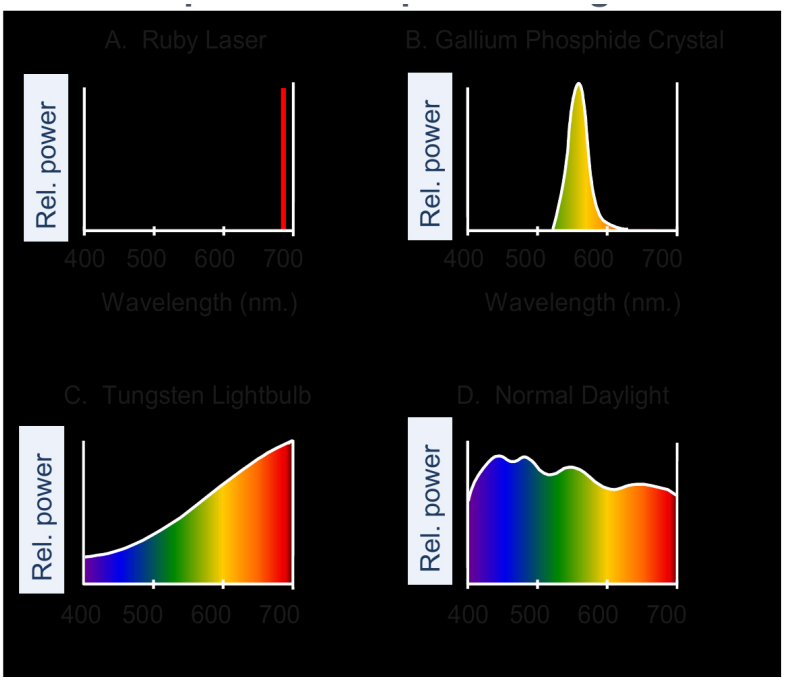


Figure 11.1. Examples from the lecture of qualitatively different source spectra, including narrow-band and broadband illumination.

A surface is similarly characterized by its reflectance spectrum: the fraction of incident light reflected at each wavelength. Therefore the observed color signal depends jointly on source spectrum and surface reflectance.

11.2 The Eye as a Sensing System

The lecture draws a camera analogy for the eye: the iris regulates the pupil aperture, the lens changes shape to focus at different distances, and the retina supplies the photoreceptors. Rods and cones are not uniformly distributed. Rods are emphasized for intensity sensitivity, while cones support color vision. The **fovea**, a

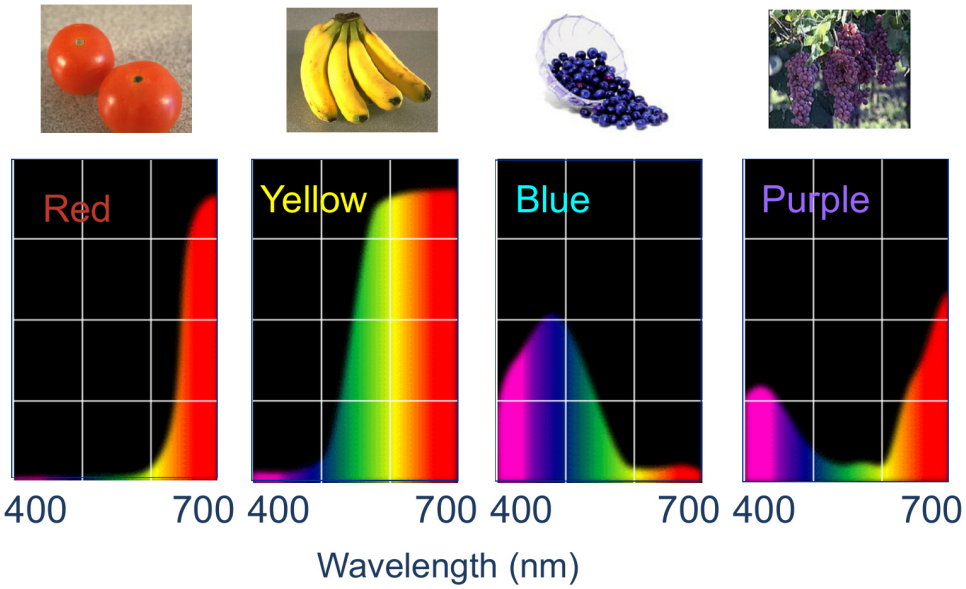


Figure 11.2. Representative surface reflectance spectra shown in the lecture for red, yellow, blue, and purple surfaces.

small central region covering roughly 1–2° of visual angle in the slides, has the highest cone density and no rods; peripheral acuity is lower because many rods can feed the same neuron.

Human color vision uses three cone classes, conventionally called S, M, and L for their relative wavelength sensitivities. The lecture gives an approximate L:M:S ratio of 10 : 5 : 1 and notes that there are almost no S cones in the center of the fovea.

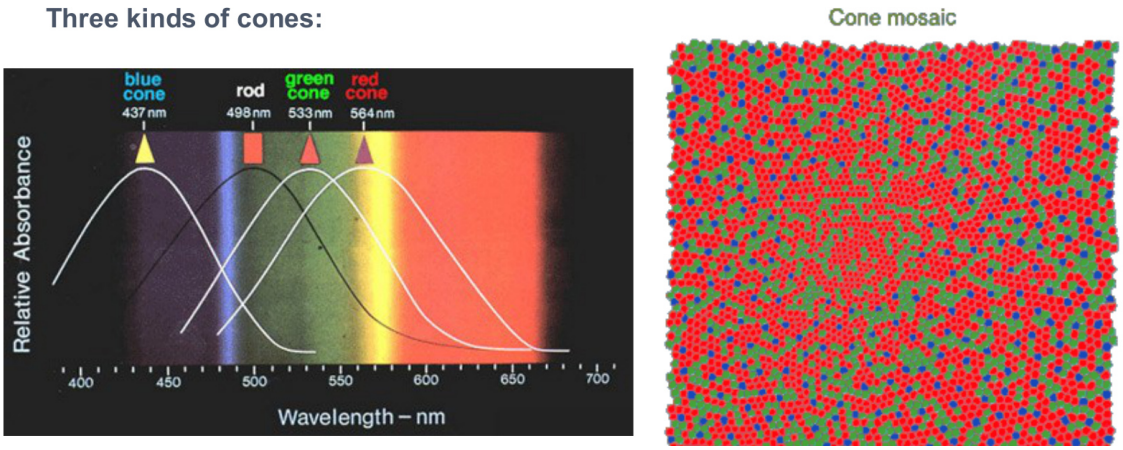


Figure 11.3. Lecture illustration of the three cone-sensitivity functions and their non-uniform representation.

Rods operate at lower illumination than cones. Below the color-perception threshold, rod-dominated vision can still detect brightness, while cone-based color and fine foveal detail become ineffective. This connects the sensitivity slide's question, "Why can't we read in the dark?", to the absence of rods in the fovea and the lower acuity of peripheral rod vision. The source chart distinguishes the threshold in bright sunlight from the much lower threshold after dark adaptation. Light/dark adaptation changes sensitivity as illumination changes; it is a gradual effect, unlike instantaneous contrast.

Brewster's-color demonstrations are used as evidence that the visual system can interpolate color from spatially offset samples. This is consistent with the fact that the three cone populations do not sample every retinal location identically.

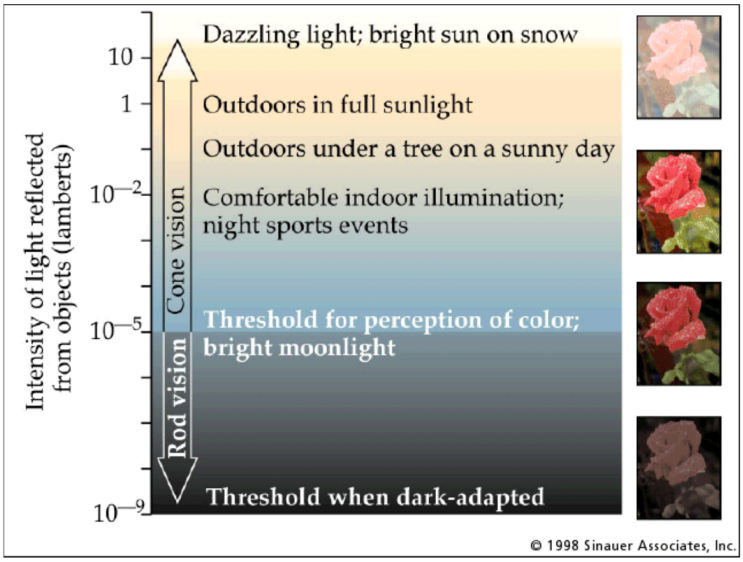


Figure 11.4. Source sensitivity regimes and light/dark adaptation thresholds. The chart is preserved rather than assigning invented numerical sensitivity curves.

11.3 Spectral Filtering and Metamers

A cone response can be understood as a spectral filter: multiply the incoming spectrum by the receptor sensitivity curve and integrate over wavelength. Each cone class therefore compresses an entire spectrum into a single response. Three cone outputs cannot preserve the full spectral signal, so distinct spectra can produce identical responses.

Metamerism

Metamers are physically different spectral power distributions that produce the same perceived color under the relevant viewing conditions. Metamerism is a direct consequence of compressing a high-dimensional spectrum into a small number of receptor responses.

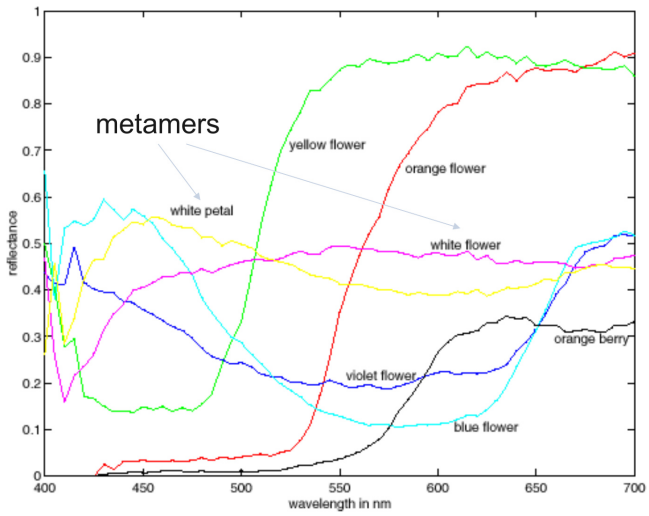


Figure 11.5. Real-world spectral curves used in the lecture to illustrate that different spectra may be perceptually matched.

Chapter summary. Physical spectra are high-dimensional, but human color perception is driven mainly by three cone classes. Consequently, color depends on both illumination and surface reflectance, and different spectra can become perceptually indistinguishable as metamers.

CHAPTER 12

Color Matching and Color Spaces

12.1 Trichromacy and Color Matching

Color-matching experiments standardize which spectra produce the same sensation under controlled conditions. The lecture's central empirical result is **trichromacy**: for most observers, three primaries are sufficient to match a wide range of test lights, with color blindness noted as an exception. The same test light and same primaries tend to produce similar matching weights across observers.

In the experiment, the observer compares a test light with a mixture of three primary lights and adjusts their weights until the two fields appear to match.

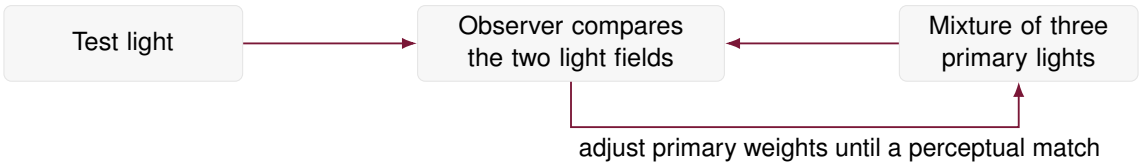


Figure 12.1. Conceptual color-matching experiment underlying trichromacy and Grassmann's laws.

12.2 Grassmann's Laws

The lecture states that additive color matching is linear. If

$$A = u_1P_1 + u_2P_2 + u_3P_3,$$
$$B = v_1P_1 + v_2P_2 + v_3P_3,$$

then mixtures and scalings obey

$$A + B = (u_1 + v_1)P_1 + (u_2 + v_2)P_2 + (u_3 + v_3)P_3, \tag{12.1}$$
$$kA = (ku_1)P_1 + (ku_2)P_2 + (ku_3)P_3. \tag{12.2}$$

where

- P_1, P_2, P_3: chosen primary lights;
- u_i, v_i: primary weights that match test lights A and B;
- k: scalar intensity multiplier.

If two test lights are matched by the same three weights, the color-matching experiment treats them as a perceptual match.

12.3 Color-Matching Functions as a Linear Transform

Choose three primaries and measure the weights $c_1(\lambda)$, $c_2(\lambda)$, and $c_3(\lambda)$ required to match monochromatic lights at sampled wavelengths. Store these color-matching functions as the rows of a matrix

$$C = \begin{bmatrix} c_1(\lambda_1) & \cdots & c_1(\lambda_N) \\ c_2(\lambda_1) & \cdots & c_2(\lambda_N) \\ c_3(\lambda_1) & \cdots & c_3(\lambda_N) \end{bmatrix}. \tag{12.3}$$

Represent a new sampled spectral signal by

$$\mathbf{t} = \begin{bmatrix} t(\lambda_1) & \cdots & t(\lambda_N) \end{bmatrix}^{\mathsf{T}}.$$

The primary coordinates are then

$$\mathbf{e} = C\mathbf{t}. \tag{12.4}$$

where

- C: $3 \times N$ matrix of sampled color-matching functions;
- $\mathbf{t}$: sampled spectral power distribution of the test light;
- $\mathbf{e} = (e_1, e_2, e_3)^{\mathsf{T}}$: three primary weights that describe its color in the chosen linear color space; $\lambda_1, \ldots, \lambda_N$ are the N sampled wavelengths, $t(\lambda_i)$ is spectral power, and $c_j(\lambda_i)$ is the matching weight for primary j.

12.4 RGB and CIE XYZ

A **linear color space** is defined by a choice of three primaries. Mixing two lights traces a straight line between their coordinates; mixtures of three primaries lie within the triangle they define. In RGB, the primaries correspond to red, green, and blue lights. The source figure specifies $p_1 = 645.2\,\text{nm}$ (red), $p_2 = 525.3\,\text{nm}$ (green), and $p_3 = 444.4\,\text{nm}$ (blue); these are the selected monochromatic primaries in that lecture example. The lecture notes that some wavelengths require subtractive matching, which appears as negative values in RGB color-matching functions.

The CIE XYZ system, standardized in 1931 by the International Commission on Illumination, uses imaginary primaries chosen so that its matching functions are nonnegative. A two-dimensional chromaticity representation is obtained by normalizing:

$$x = \frac{X}{X+Y+Z}, \qquad y = \frac{Y}{X+Y+Z}. \tag{12.5}$$

where

- X, Y, Z: CIE tristimulus coordinates;
- x, y: normalized chromaticity coordinates that remove the common scale $X + Y + Z$ (defined when this sum is nonzero).

12.5 Toward Perceptual Uniformity

The lecture stresses that Euclidean differences in CIE x, y coordinates are not perceptually uniform. MacAdam ellipses illustrate that just-noticeable color differences occupy differently sized and oriented regions of the chromaticity diagram. CIE $u'v'$ is introduced as a projective transform intended to make these regions more uniform, followed by CIE $L^*a^*b^*$ as a later perceptually motivated space.

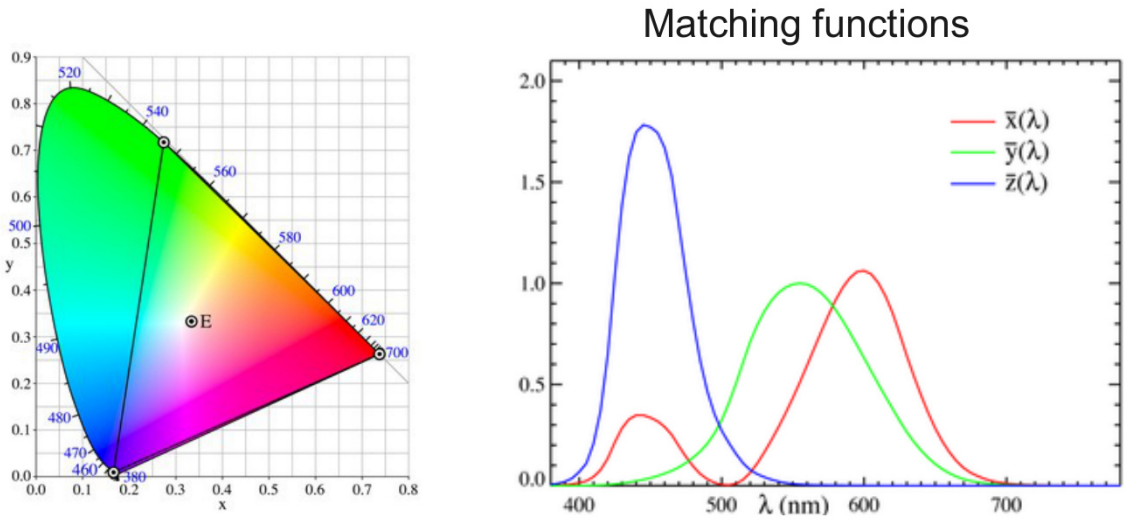
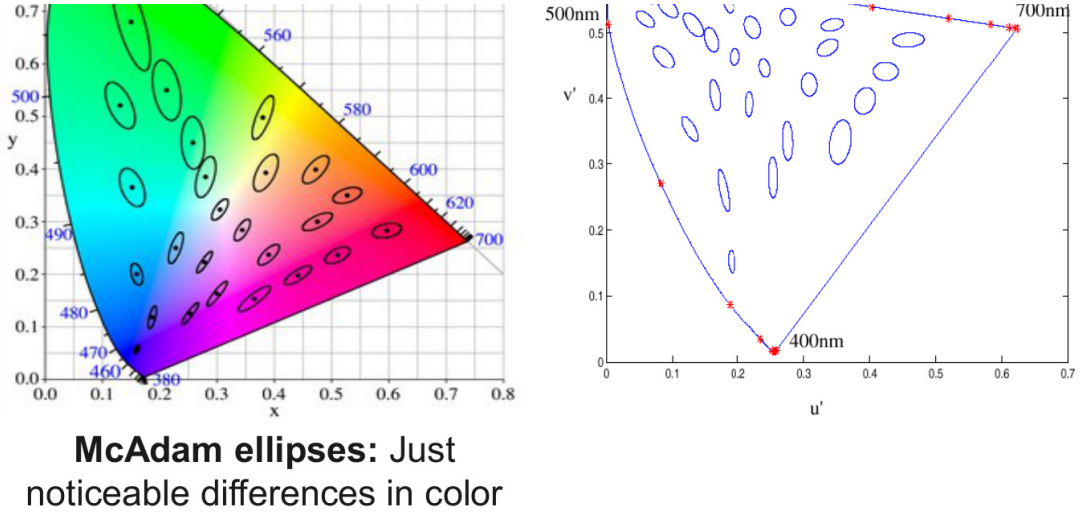


Figure 12.2. CIE XYZ material from the lecture: chromaticity visualization and the associated nonnegative matching functions.



McAdam ellipses: Just noticeable differences in color

Figure 12.3. MacAdam-ellipse illustration used to motivate more perceptually uniform color coordinates.

12.6 HSV

HSV reorganizes color into perceptually interpretable dimensions: **Hue** answers "what color?", **Saturation** describes how much chromatic purity is present, and **Value** describes brightness. The lecture visualizes the model as a cylinder or cone and interprets it geometrically as a reorganization of the RGB cube. In the source's geometric construction, the RGB cube is tilted onto its black–white diagonal: that diagonal becomes the brightness axis, while chromatic colors lie around it. Hue is the angular direction around this axis, saturation moves outward from the neutral axis, and value moves along the brightness direction. The source then schematically places hues in a common plane, expands the cube, and smooths it toward a cylinder. This representation makes operations such as shifting hue or increasing saturation conceptually simple.

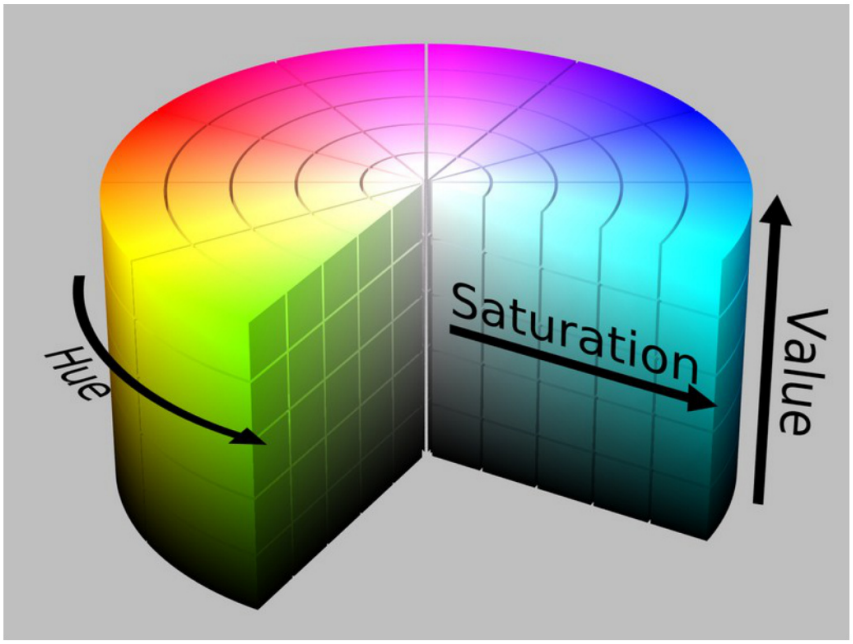


Figure 12.4. Geometric HSV representation shown in the lecture, emphasizing hue, saturation, and value as different axes of description.

Chapter summary. Trichromacy makes three-coordinate color spaces possible. Grassmann's laws justify linear color matching, while $\mathbf{e} = C\mathbf{t}$ maps a sampled spectrum into primary coordinates. RGB is device-oriented, CIE XYZ provides a standardized linear basis, CIE $u'v'$ and $L^*a^*b^*$ address perceptual uniformity, and HSV reorganizes RGB-like color into perceptual dimensions.

CHAPTER 13

Color Constancy and White Balance

13.1 Constancy and Context

Color/lightness constancy is the ability of the human visual system to perceive relatively stable surface reflectance despite changes in illumination. The lecture separates several classes of perceptual effects:

- **instantaneous**: simultaneous contrast and Mach bands;
- **gradual**: light/dark adaptation, chromatic adaptation, and afterimages;
- **high-level**: effects related to color naming and scene interpretation.

Simultaneous contrast means that a target's apparent lightness or color changes with its surrounding field. **Mach bands** emphasize apparent bright and dark bands near transitions between neighboring intensity regions: perceived edge contrast can exceed the physical change.

The checker-shadow illusion shows that equal measured image intensities can be perceived differently. A boundary between checker squares suggests a **reflectance change** (different surface material), whereas the cast-shadow boundary suggests an **illumination change** across a surface. Interpreting the shadow allows the observer to judge surface lightness rather than treating every intensity change as a change of reflectance.

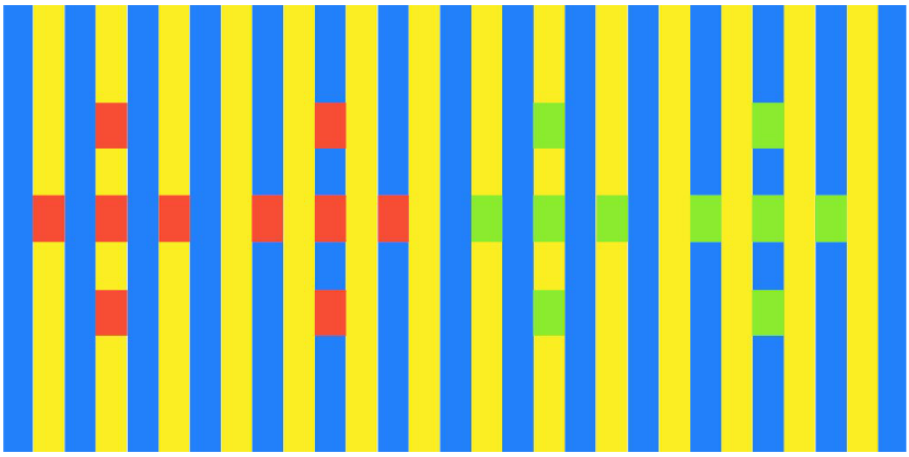


Figure 13.1. The source's simultaneous-contrast/Mach-band demonstration. The colored targets are viewed within alternating surrounding stripes; the perceptual artwork is preserved.

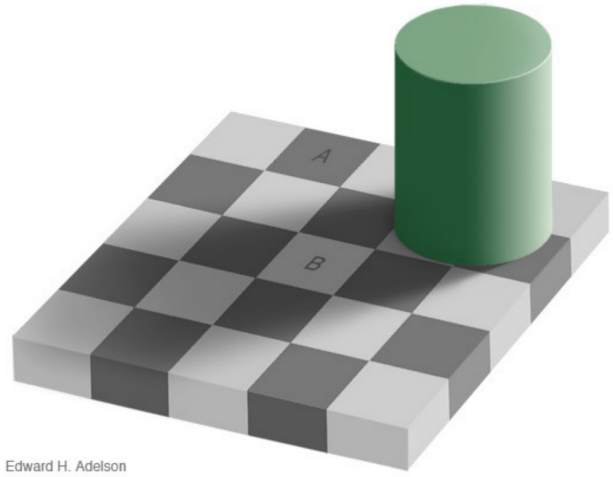


Figure 13.2. Checker-shadow example from the lecture, used to discuss lightness constancy and the distinction between reflectance and illumination changes.

13.2 Chromatic Adaptation

The visual system changes sensitivity according to the prevailing luminance and chromatic conditions. Aperture changes regulate total light entering the eye, while receptor sensitivity also adapts. The lecture's example is that under increased red illumination, red-sensitive responses are gradually reduced until the scene appears more neutral. Adaptation works better in brighter scenes, which helps explain why candlelit scenes may still look yellow.

> **Human perception versus a computer**
>
> Humans use context across the entire scene and perform strong perceptual normalization. A computer ordinarily starts from measured pixel values and does not automatically separate an object's intrinsic color from illumination. Algorithms such as white balance attempt to approximate part of this adaptation.

13.3 White Balance

When a photograph is viewed on a screen or print, the observer adapts to the room's illuminant rather than the illuminant present in the photographed scene. Incorrect camera white balance therefore appears as an unnatural color cast.



Figure 13.3. Incorrect and corrected white balance as illustrated in the lecture.

The lecture contrasts film and digital practice. Film cameras use different films or filters for different illuminants, whereas digital cameras can use automatic white balance, predefined illuminant settings, or a custom neutral reference. Multiple illuminants make the task ambiguous because different reference regions can imply different corrections.

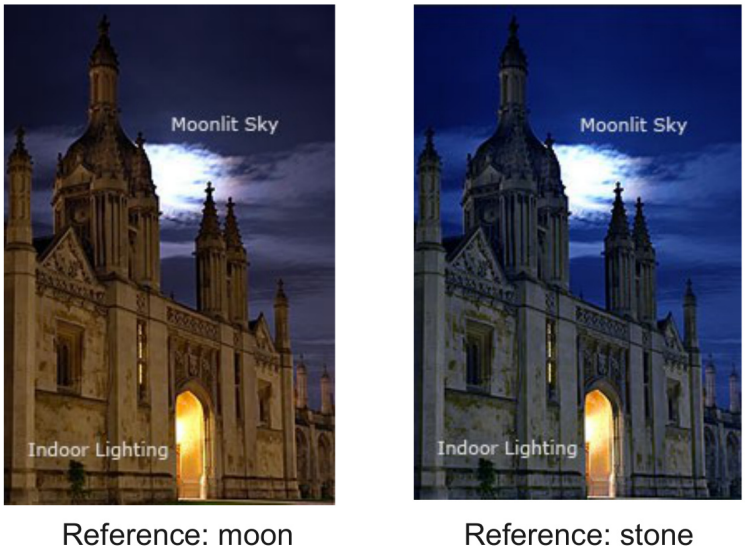


Figure 13.4. Mixed illumination: selecting different neutral references can lead to different white-balance results.

13.3.1 Von Kries Adaptation and a Gray Card

The lecture presents a channel-gain model: multiply each channel by a separate factor. If a known neutral reference is recorded as (r_w, g_w, b_w), choose gains proportional to

$$g_R = \frac{1}{r_w}, \qquad g_G = \frac{1}{g_w}, \qquad g_B = \frac{1}{b_w}. \tag{13.1}$$

where

- (r_w, g_w, b_w): measured RGB values of a white or gray reference object;
- g_R, g_G, g_B: per-channel multiplicative gains (the reference channels must be nonzero).

The slides also note that a light source can have a more complex effect than independent channel scaling; in general it may be represented by a full 3×3 transformation.

13.3.2 White Balance without a Gray Card

When no neutral reference is provided, the camera must infer the illuminant. The lecture lists several strategies:

- **gray-world assumption**: assume the image averages $(r_{ave}, g_{ave}, b_{ave})$ should be gray and use inverse-average gains;
- **brightest-pixel assumption**: highlights often approximate the illuminant color, so use gains inversely proportional to the brightest channel values;
- **gamut mapping**: match the convex hull of observed image colors to a typical gamut under white light;
- **image statistics and learning**: estimate illuminant from learned regularities.

A higher-level method cited in the lecture performs **white balance by recognition**: for semantic classes detected in the image, estimate the illuminant that maps the class's measured average color toward the average color of that class in a database of typical images (Van de Weijer, Schmid, and Verbeek, ICCV 2007).

> **Chapter summary.** Color constancy is a perceptual normalization problem; white balance is its computational analogue in imaging. Simple methods use channel gains and neutral references, while gray-world, brightest-pixel, gamut, statistical, and semantic-class methods estimate the illuminant when no reference card is available.

CHAPTER 14

Color in Computer Vision and Texture

14.1 Uses of Color

The lecture surveys several uses of color in computer vision and associates them with representative research examples:

14.2 Structural and Statistical Texture

The lecture defines **texture** as a description of the spatial arrangement of color or intensities in an image or selected region. A structural approach models a repeated arrangement of basic texture elements (texels). Natural images often make explicit texel extraction difficult or impossible, motivating statistical texture descriptions computed directly from gray tones or colors.

> **Why statistical texture?**
>
> Statistical texture descriptors are less intuitive than explicit texel models but are computationally efficient and can support both classification and segmentation. They are especially useful when natural texture elements cannot be segmented reliably.

Table 14.1. Color-based computer-vision applications and references cited in the lecture.

Application	Representative source named in the lecture
Color histograms for indexing and retrieval	Swain and Ballard, *Color Indexing*, IJCV 1991
Skin detection	Jones and Rehg, *Statistical Color Models with Application to Skin Detection*, IJCV 2002
Image segmentation and retrieval	Carson, Belongie, Greenspan, and Malik, *Blobworld*, 1999
Robot soccer	Sridharan and Stone, color calibration at RoboCup, 2006
Appearance models for tracking	Ramanan, Forsyth, and Zisserman, *Tracking People by Learning their Appearance*, PAMI 2007
Judging visual realism	Lalonde and Efros, color compatibility for image realism, ICCV 2007

14.3 Local Binary Patterns

For each pixel p, the Local Binary Pattern (LBP) example in the lecture compares the eight neighbors with the center value. For neighbor i,

$$b_i = \begin{cases} 0, & \text{if neighbor } i \text{ has value} \le p, \\ 1, & \text{otherwise,} \end{cases} \tag{14.1}$$

Here p denotes the center intensity in this comparison, $i = 1, \ldots, 8$ follows the source neighbor ordering, and b_i is the resulting comparison bit. Then the eight bits $b_1 \ldots b_8$ form a local code. A region is represented by a histogram of these codes.

LBP lecture example

For the 3×3 neighborhood

$$\begin{bmatrix} 100 & 101 & 103 \\ 40 & 50 & 80 \\ 50 & 60 & 90 \end{bmatrix},$$

the center pixel is 50. Starting at the upper-left neighbor and proceeding clockwise gives $100, 101, 103, 80, 90, 60, 50, 40$. Comparing the eight neighbors with the center produces the bit pattern shown in the lecture, 11111100, under its stated neighbor ordering. The histogram of such codes becomes a texture descriptor.

The slides show LBP-histogram retrieval in the Fids image database and explicitly warn that low-level similarity does not always imply semantic similarity. Additional examples mentioned include deciduous-tree classification, finding cancer in Pap smears, and identifying regions of interest in breast and melanoma biopsy slides.

14.4 Image Data Structure

The lecture closes with a simple implementation-oriented image representation:

```
typedef struct{
    int h, w, c;
    float *data;
} image;
```

Here h is height, w is width, and c is the number of channels ($c = 3$ for RGB and $c = 1$ for grayscale in the example). The image data are stored as a linear array of floating-point values in $[0, 1]$, i.e., a three-dimensional image tensor is flattened into one-dimensional memory.

> **Chapter summary.** Color supports retrieval, segmentation, detection, tracking, calibration, and realism assessment. Texture describes spatial appearance; statistical descriptors such as LBP summarize local comparisons as histograms when explicit texel extraction is impractical.

Lecture 3 Quick Review

Light and human vision

1. Color is a perceptual result of physical light interacting with the visual system.
2. Source spectra and surface reflectance jointly determine the spectral signal reaching the observer.
3. Rods are emphasized for intensity sensitivity; three cone classes support color vision.
4. Metamers have different spectra but the same perceived color under the relevant conditions.
5. Trichromacy allows color to be encoded by three primary weights.

Color matching and spaces

6. Grassmann's laws make additive color matching linear.
7. Color matching can be written compactly as $\mathbf{e} = C\mathbf{t}$.
8. RGB is a linear primary space; some spectral matches require subtractive components.
9. CIE XYZ uses nonnegative matching functions and chromaticity coordinates x, y.
10. MacAdam ellipses show that equal x, y distances are not equal perceptual differences.
11. HSV separates hue, saturation, and value for perceptually meaningful manipulation.

Constancy and appearance

12. Color/lightness constancy depends on context and adaptation.
13. White balance compensates for illuminant color; a gray card provides a direct neutral reference.
14. Gray-world, brightest-pixel, gamut, and learning methods estimate white balance without a reference card.
15. Texture describes spatial organization; statistical descriptors avoid explicit texel segmentation.
16. LBP compares neighbors with a center pixel and summarizes the resulting codes with a histogram.

LECTURE 4

Histograms and Intensity Transformations

From intensity distributions to contrast enhancement and normalization.

Histograms · Equalization · Projection · Contrast · Applications

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 15

Histograms and Point Transformations

A histogram summarizes how image values are distributed. In this lecture the histogram is used not only as a diagnostic representation of brightness and contrast, but also as the basis for a family of point transformations that remap each input gray level to a new output level.

15.1 Intensity Histograms

For an intensity image, the histogram records how many pixels take each available gray level. The lecture uses the usual display convention in which 0 denotes black and 255 denotes white for an 8-bit image.

Histogram lecture example

The source example uses the 4×5 image

$$\begin{bmatrix} 0 & 1 & 1 & 2 & 4 \\ 2 & 1 & 0 & 0 & 2 \\ 5 & 2 & 0 & 0 & 4 \\ 1 & 1 & 2 & 4 & 1 \end{bmatrix}.$$

Its histogram counts how many times each gray level occurs; in particular the visible bars correspond to counts $5, 6, 5, 0, 3, 1$ for levels $0, 1, 2, 3, 4, 5$.

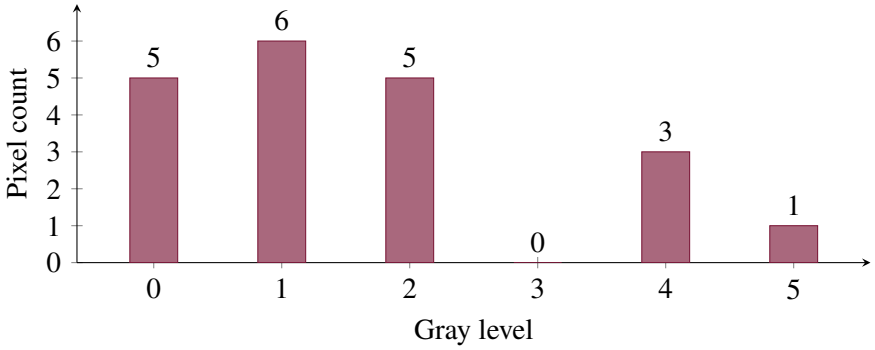


Figure 15.1. Histogram reconstructed from the displayed 4×5 image. The counts sum to 20 pixels.

A histogram is also a direct indicator of contrast. When only a small portion of the available range is occupied, the image has low contrast. The source illustrates this with a gray image whose histogram is concentrated in a narrow interval.

15.2 Histograms of Color Images

The lecture presents three complementary ways to form histograms from color data. First, RGB can be converted to a gray-scale brightness value using

$$Y = 0.299R + 0.587G + 0.114B. \tag{15.1}$$

where

- R, G, B: red, green, and blue channel values;
- Y: gray-scale component in the YIQ color space used in NTSC television;
- the coefficients: weights reflecting the eye's relative brightness sensitivity to the three primaries, as stated in the lecture.

Second, one may compute a separate one-dimensional histogram for each of the red, green, and blue channels. Third, the source notes that a three-dimensional histogram can be formed in RGB space: the three axes represent red, green, and blue, while the value at a location represents the number of pixels with that color combination.

15.3 Point Transformations

Histogram-based enhancement is implemented as a point operation

$$s_k = T(r_k), \tag{15.2}$$

where

- r_k: the k-th input gray level;
- s_k: transformed output gray level;
- T: point transformation applied to gray levels.

The source requires two properties of T:

- it keeps values within the original gray-level range;
- it is monotonically nondecreasing, so the ordering of gray levels is not reversed; distinct input levels may map to the same output level.

> **Why the histogram matters**
>
> A point transformation does not inspect spatial neighborhoods. It changes a pixel only from its current gray level. The histogram therefore supplies exactly the global information needed to design or evaluate many such mappings.

> **Chapter summary.** A histogram counts the occurrence of image values and reveals whether the available intensity range is being used effectively. Color images can be summarized through luminance, separate RGB histograms, or a multidimensional color histogram. Histogram enhancement then applies a monotone point mapping $s_k = T(r_k)$ to redistribute those values.

CHAPTER 16

Histogram Equalization, Projection, and Plateau Methods

16.1 Histogram Equalization

Histogram equalization (HE) transforms image intensities so that the output histogram approximately approaches a flat or uniform distribution. In the discrete formulation shown in the lecture,

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k}\frac{n_j}{n}, \qquad k = 0, 1, \ldots, L-1. \tag{16.1}$$

where

- r_k: the k-th input gray level;
- s_k: equalized output gray level;
- L: number of gray levels in the lecture notation;
- n_j: number of pixels having the j-th gray level;
- n: total number of image pixels;
- $\sum_{j=0}^{k} n_j/n$: cumulative fraction of pixels at or below level k.

> **Source notation for L**
>
> Here L is the number of gray levels: an 8-bit image has $L = 256$ levels, indexed from 0 to $L-1 = 255$. The slide's example "255" denotes the maximum index, not the number of levels. Round the transformed values to valid integer output levels in a discrete implementation.

The cumulative histogram is central: as k increases, the cumulative fraction grows from 0 toward 1, and the transformation spreads frequently occupied input ranges across more of the output interval. Because the image is discrete, the final histogram is only approximately uniform.

The lecture also emphasizes **local equalization**: the histogram can be computed over only part of an image rather than globally. This allows enhancement to adapt to local contrast conditions.

16.2 Histogram Projection

Histogram projection (HP) assigns equal display space to every *occupied* raw signal level, regardless of how many pixels use that level. It therefore depends on which levels are present rather than on their frequencies.

A gray level is occupied if at least one image pixel has that value. Let $B(k)$ denote the fraction of occupied levels at or below level k. If the number of occupied levels is N_{occ}, then $B(k)$ rises from 0 to 1 in uniform steps of $1/N_{\text{occ}}$. The lecture's display transform is

$$s_k = 255\, B(k). \tag{16.2}$$

where

- $B(k)$: cumulative fraction of occupied gray levels at or below k;
- N_{occ}: total number of occupied levels; the slide denotes this quantity by n;
- s_k: display value assigned to raw level k on the 0–255 scale.

> **HE and HP use different counts**
>
> In the HE formula, n denotes the total number of pixels. In the HP slide, n denotes the number of occupied gray levels. The two transformations therefore weight the histogram in fundamentally different ways.

16.3 Plateau Equalization

Plateau equalization (PE) clips histogram counts at a chosen plateau (or saturation) value before forming the cumulative distribution. The resulting display allocation lies between HE and HP.

The source identifies the limiting cases explicitly:

- plateau value = 1: the clipped histogram records only whether a level is occupied, producing $B(k)$ and therefore HP;
- plateau value above the histogram peak: clipping has no effect, producing the ordinary cumulative distribution $F(k)$ and therefore HE;
- intermediate plateau values: produce an intermediate cumulative distribution denoted $P(k)$.

The PE transform is

$$s_k = 255\,P(k), \tag{16.3}$$

where

- $P(k)$: cumulative distribution obtained after clipping histogram counts at the selected plateau value;
- s_k: resulting output display value on the 0–255 scale.

> **Chapter summary.** HE distributes display values according to cumulative pixel frequency, HP distributes them according to occupied gray levels, and PE interpolates between those behaviors by clipping histogram counts before forming the cumulative distribution. Local equalization applies the same idea to a selected image region.

CHAPTER 17

Histogram Specification and Contrast Stretching

17.1 Histogram Specification

Histogram specification (HS), also called histogram matching in the lecture's normalization discussion, transforms an image so that its output histogram approximately matches a desired histogram rather than a uniform one.

Let T denote the cumulative transform of the source histogram and let S denote the cumulative transform associated with the desired histogram. The source diagram expresses the mapping as

$$z = S^{-1}(T(r)), \tag{17.1}$$

where

- r: original image intensity;
- T: cumulative transform of the source histogram;
- S: cumulative transform associated with the desired histogram;
- z: output intensity after applying the inverse target transform.



Figure 17.1. Histogram specification: transform through the source cumulative mapping and the inverse target mapping, corresponding to the lecture's $S^{-1}T$ diagram.

17.2 Contrast Stretching

Contrast stretching (CS) attempts to use the available full gray-level range by expanding a selected input interval. The source gives

$$s_k = 255\,\frac{r_k - r_{\min}}{r_{\max} - r_{\min}}. \tag{17.2}$$

where

- r_k: input gray level;
- $r_{\min}, r_{\max}$: lower and upper values selected for the stretch;
- s_k: output display value on the 0–255 range.

This formula requires $r_{\max} > r_{\min}$. Clamp values outside the selected interval to 0 or 255; a constant image needs an explicit output convention because its minimum and maximum coincide. The same idea can map one intensity interval to another output interval.

The examples show that a simple minimum–maximum stretch is not always enough: when the histogram shape itself is strongly nonuniform, HE can reveal information that CS does not. Conversely, a carefully chosen stretch can preserve a more natural appearance than full equalization.

The source presents several ways to choose the effective input interval:

- use nearly the full distribution, e.g. the 1%–99% percentile range;
- choose explicit limits, illustrated by the pair 79, 136;
- use a **cutoff fraction**, with the slide example 0.8: the cutoff fraction is the minimum fraction of the histogram peak magnitude below which data are ignored when selecting the stretch range.

> **CS versus HE**
>
> Contrast stretching expands a chosen input interval by a direct point mapping. Histogram equalization uses the cumulative histogram and therefore changes the allocation according to how pixels are distributed. The two methods can produce noticeably different results on the same image.

> **Chapter summary.** Histogram specification matches a chosen target distribution using the source and target cumulative transforms. Contrast stretching instead maps a selected input interval across a wider output range. Percentile, explicit-limit, and cutoff-based strategies reduce the influence of extreme or uninformative histogram regions.

CHAPTER 18

Applications of Histogram Processing

18.1 Thresholding

Histogram inspection can support conversion from gray-scale to binary imagery. In a clearly bimodal histogram, a threshold between the two modes can separate the image into two classes. The source's bimodal example uses a threshold of 120.

When the histogram is not bimodal, the choice is less direct. The lecture compares thresholds 80 and 120 on the same example to show that different choices can produce substantially different binary segmentations.

18.2 Normalization and Histogram Matching

When images are to be compared using properties such as texture, the lecture recommends normalizing their histograms to a standard distribution, especially when the images were acquired under different conditions. Histogram equalization is one simple normalization. Histogram matching goes further by taking into account the shapes of both the original histogram and the histogram being matched.

18.3 Medical-Image Examples

The lecture uses CT and MRI to illustrate why histogram processing is useful in practice. In CT lung studies, equalization can be computed over a selected region rather than the full image, emphasizing local diagnostic detail.

MRI intensities do not have a fixed physical meaning in the way the displayed gray value might suggest.

The source states that this is true even for the same protocol, body region, scanner, and patient. Intensity standardization therefore attempts to make images more comparable.

The lecture reproduces the Nyúl–Udupa example: panel A shows histograms of ten FSE proton-density brain-volume images of multiple-sclerosis patients, panel B shows those histograms after scaling, and panel C shows them after final standardization.

18.4 High-Dynamic-Range Display

Digitized high-dynamic-range data such as infrared images and CT/CAT images may span a much larger raw range than the 0–255 levels available on a conventional monitor. A display algorithm must therefore map the raw signal into 0–255 while preserving information that is useful to the human observer.

The source notes two operational examples:

- HP is widely used by infrared camera manufacturers for real-time automated display;
- PE is used in the B-52 infrared navigation and targeting sensor.

> **Chapter summary.** Histogram methods support thresholding, normalization, medical-image standardization, local contrast enhancement, and display of high-dynamic-range data. Their usefulness comes from adapting the available display range to the distribution that matters for the task.

Lecture 4 Quick Review

Histogram foundations

1. A histogram counts how many pixels take each gray level and exposes unused intensity range.
2. RGB may be converted to brightness by $Y = 0.299R + 0.587G + 0.114B$, or analyzed with separate/multidimensional color histograms.
3. Histogram point processing uses a monotone mapping $s_k = T(r_k)$ within the available range.

Equalization family

4. HE maps cumulative pixel frequency across the display range and can also be applied locally.
5. HP assigns equal display space to every occupied gray level, independent of its pixel count.
6. PE clips histogram peaks: plateau 1 gives HP; a plateau above the peak gives HE.

Specification and contrast

7. Histogram specification uses $S^{-1}(T(r))$ to approximately match a target distribution.
8. Contrast stretching maps a selected interval to a wider range; the source also uses percentile, fixed-limit, and cutoff-fraction choices.
9. CS and HE need not produce the same enhancement because only HE uses cumulative pixel frequency.

Applications

10. Histogram structure can guide threshold selection; non-bimodal cases are more ambiguous.
11. Normalization makes differently acquired images more comparable; histogram matching accounts for both histogram shapes.
12. MRI intensity standardization, CT local enhancement, and IR/CT display are representative applications.

LECTURE 5

Spatial Filters and Image Enhancement

From local neighborhoods to smoothing, sharpening, and combined enhancement.

Neighborhoods · Smoothing · Laplacian · Sobel · Enhancement

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 19

Spatial Filtering Foundations

Lecture 5 moves from point operations to **neighborhood operations**. Instead of transforming a pixel from its value alone, a spatial filter computes the output at (x, y) from a neighborhood around that location. The source follows the spatial-filtering material of Gonzalez and Woods, *Digital Image Processing*, Chapter 3.

19.1 Neighborhood Operations

A neighborhood is typically a rectangular region centered on the current pixel, although the lecture notes that other sizes and shapes are possible. For each pixel of the original image, an output is written at the corresponding location in the target image.

Two simple non-linear neighborhood examples are:

- **minimum filter**: replace the center pixel by the minimum value in its neighborhood;
- **maximum filter**: replace the center pixel by the maximum value in its neighborhood.

19.2 The Spatial Filtering Process

For a filter of size $(2a + 1) \times (2b + 1)$, the lecture gives the correlation-style spatial filtering equation

$$g(x, y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s, t) f(x + s, y + t). \tag{19.1}$$

where

- $f(x, y)$: input image;
- $g(x, y)$: filtered output image;
- $w(s, t)$: filter/kernel coefficient at offset (s, t);
- a, b: half-sizes of the neighborhood in the two directions;
- s, t: offsets relative to the current pixel.

The operation is repeated at every pixel location. The source's 3×3 diagram writes the processed center as the sum of the nine neighboring image samples multiplied by the corresponding nine kernel entries.

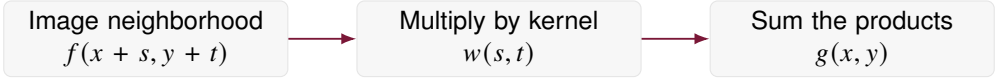


Figure 19.1. Correlation at one pixel: multiply corresponding samples and kernel weights, then sum. Repeat at every output location.

19.3 Correlation and Convolution

The filtering operation used up to this point is called **correlation**, and the filter is a correlation kernel. **Convolution** is the closely related operation obtained by reversing the kernel before applying the same local multiply-and-sum process. The lecture emphasizes that the distinction disappears for symmetric filters.

> **Correlation versus convolution**
>
> If a kernel is symmetric under the required reversal, correlation and convolution produce the same result. This is why many smoothing kernels behave identically under the two operations even though the formal definitions differ.

19.4 Image Borders

At an image boundary, a full neighborhood may extend beyond the available pixels. The source lists four strategies:

- omit missing pixels when the filter permits it;
- pad the image, typically with all-white or all-black pixels;
- replicate border pixels;
- truncate the output image.

> **Chapter summary.** Spatial filtering computes each output pixel from a local neighborhood. Correlation applies the kernel as written; convolution reverses it first, with no difference for symmetric kernels. Boundary handling must be specified because complete neighborhoods are unavailable at image edges.

CHAPTER 20

Smoothing Spatial Filters

20.1 Averaging Filters

The simplest smoothing filter replaces the center pixel by the average of the pixels in its neighborhood. For a 3×3 neighborhood, the source uses

$$W_{\text{mean}} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

Smoothing is useful for reducing noise and for suppressing fine detail so that large-scale structures become more prominent.

Numerical averaging example from the lecture

For the neighborhood

$$\begin{bmatrix} 104 & 100 & 108 \\ 99 & 106 & 98 \\ 95 & 90 & 85 \end{bmatrix},$$

the mean-filter output at the center is

$$\frac{104 + 100 + 108 + 99 + 106 + 98 + 95 + 90 + 85}{9} = 98.3333.$$

The same computation is repeated over the image.

As the averaging-filter size increases, more fine detail disappears. The source compares filter sizes 3, 5, 9, 15, and 35 on a 500×500 test image.

20.2 Weighted Smoothing

More selective smoothing assigns different importance to different neighbors. The lecture's weighted averaging example is

$$W_{\text{weighted}} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

Pixels closer to the center receive greater weight. The coefficients sum to one, so a constant region remains unchanged by the filter.

20.3 Smoothing as Preprocessing

By removing fine detail, smoothing can simplify a later decision such as thresholding. The source illustrates this with an astronomical image: smoothing suppresses small bright structures and leaves the gross objects that are then isolated by thresholding.

20.4 Averaging and Median Filtering

Filtering is frequently used to remove noise, but the lecture stresses that an averaging filter is not always the best choice. Its comparison shows that a **median filter** can preserve structures while removing impulsive-looking noise more effectively than a simple average.

Chapter summary. Averaging filters reduce noise and fine structure by local averaging, while weighted smoothing gives greater influence to selected neighbors. Increasing the support size produces stronger blur. Median filtering is a non-linear alternative that can outperform averaging on some noise patterns.

CHAPTER 21

Sharpening and Spatial Derivatives

Sharpening filters seek to emphasize fine detail, reduce the visual effect of blur, and highlight edges. The lecture develops them from spatial differentiation.

21.1 First and Second Discrete Derivatives

For a one-dimensional function, the source uses the forward first difference

$$\frac{\partial f}{\partial x} = f(x+1) - f(x), \tag{21.1}$$

and the second difference

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x). \tag{21.2}$$

where

- $f(x)$: one-dimensional sampled signal;
- x: sample position;
- $\partial f/\partial x$: first discrete difference used in the lecture;
- $\partial^2 f/\partial x^2$: second discrete difference.

The first derivative measures the change between successive values, while the second derivative compares the current value with both its predecessor and successor.

The source summarizes the practical differences as follows:

- first-order derivatives generally produce thicker edges;
- second-order derivatives respond more strongly to fine detail such as thin lines;
- first-order derivatives have a stronger response to a gray-level step;
- second-order derivatives produce a double response at a gray-level step.

The second derivative is emphasized for image enhancement because of its strong response to fine detail and simple implementation.

21.2 The Laplacian

The Laplacian is the isotropic second-order operator

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}. \tag{21.3}$$

With the source's discrete second differences,

$$\begin{aligned} \nabla^2 f(x,y) = {} & f(x+1,y) + f(x-1,y) \\ & + f(x,y+1) + f(x,y-1) - 4f(x,y). \end{aligned} \tag{21.4}$$

where

- $f(x, y)$: image intensity at spatial coordinates (x, y);
- $\partial^2 f/\partial x^2$ and $\partial^2 f/\partial y^2$: second differences in the horizontal and vertical directions;
- $\nabla^2 f$: Laplacian response obtained by summing those second-order components.

This corresponds to the kernel

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Applying the Laplacian alone produces an image dominated by edges and other discontinuities; it is therefore not yet the final enhanced image. With the sign convention above, the lecture sharpens by subtracting the Laplacian response:

$$g(x, y) = f(x, y) - \nabla^2 f(x, y). \tag{21.5}$$

where

- $f(x, y)$: original image;
- $\nabla^2 f(x, y)$: Laplacian response;
- $g(x, y)$: sharpened image.

The subtraction can be combined into one spatial filter:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

The source also shows the 8-neighbor Laplacian variant

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

and its corresponding one-step sharpening kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}.$$

21.3 Unsharp Masking and Highboost Filtering

The lecture presents unsharp masking as a sequence of linear spatial operations:

1. blur the original image;
2. subtract the blurred image from the original to obtain an unsharp mask;
3. add the mask back to the original to obtain the sharpened image.

The following source illustration also compares the result with a stronger highboost-filtered image.

21.4 First-Derivative Filtering and the Gradient

For a two-dimensional image, the gradient is

$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix}. \tag{21.6}$$

Its magnitude is

$$\|\nabla f\| = \sqrt{G_x^2 + G_y^2}, \tag{21.7}$$

and the lecture gives the practical approximation

$$\|\nabla f\| \approx |G_x| + |G_y|. \tag{21.8}$$

where

- $G_x = \partial f/\partial x$: horizontal gradient component;
- $G_y = \partial f/\partial y$: vertical gradient component;
- ∇f: gradient vector;
- $\|\nabla f\|$: gradient magnitude; the lecture also uses the L_1-style approximation $|G_x| + |G_y|$.

Using the 3×3 neighborhood

$$\begin{bmatrix} z_1 & z_2 & z_3 \\ z_4 & z_5 & z_6 \\ z_7 & z_8 & z_9 \end{bmatrix},$$

the source writes the magnitude approximation as

$$\begin{aligned} \|\nabla f\| \approx\ & |(z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)| \\ & + |(z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)|\,. \end{aligned} \tag{21.9}$$

where

- $z_1, \ldots, z_9$: the nine image samples in the displayed 3×3 neighborhood;
- the first absolute difference: one Sobel directional response;
- the second absolute difference: the orthogonal Sobel directional response.

21.5 Sobel Operators

The preceding expression leads to the two Sobel masks shown in the lecture:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, \qquad \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}.$$

The image is filtered with both operators and the two responses are combined. Sobel filtering is typically used for edge detection.

> **Chapter summary.** Sharpening is based on spatial derivatives. The Laplacian uses second differences and can be subtracted from the image for sharpening; unsharp masking forms a high-frequency mask by subtracting a blur; and first-derivative methods use the image gradient, with Sobel

masks providing a practical approximation to horizontal and vertical changes.

CHAPTER 22

Combining Spatial Enhancement Methods

The final lecture example stresses that successful enhancement is typically not achieved by a single operator. Several filters and point transformations can be combined so that one stage supplies information needed by the next.

22.1 Bone-Scan Enhancement Pipeline

The source combines Laplacian sharpening, Sobel edges, smoothing, masking, and a final power-law transformation on a bone scan. The illustrated stages are:

1. **(a)** original bone scan;
2. **(b)** Laplacian-filtered version of (a);
3. **(c)** sharpened image obtained by subtracting the Laplacian result from the original;
4. **(d)** Sobel-filtered version of (a);
5. **(e)** image (d) smoothed with a 5×5 averaging filter;
6. **(f)** product of the sharpened image (c) and the smoothed Sobel image (e), used as a mask;
7. **(g)** sharpened image formed as the sum of the original (a) and mask (f);
8. **(h)** result of applying a power-law transformation to (g).

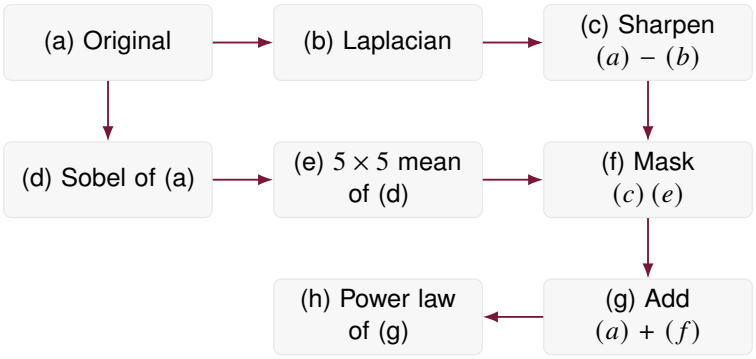


Figure 22.1. Dependencies of the bone-scan enhancement stages described above. The original (a) also supplies the subtraction in (c) and addition in (g).

Enhancement as a pipeline

The example demonstrates the purpose of combining operators: the Laplacian supplies fine-detail sharpening, the Sobel operator supplies edge information, smoothing stabilizes that edge map, multiplication forms a spatially selective mask, and the final point transformation adjusts the visual result.

Chapter summary. Spatial enhancement can combine complementary operations rather than relying on one filter. The bone-scan example uses Laplacian sharpening, Sobel edge detection, averaging, multiplicative masking, addition to the original, and a final power-law transformation to obtain the final image.

Lecture 5 Quick Review

Spatial filtering

1. Neighborhood operations use local pixels rather than a single input value.
2. Linear spatial filtering computes a weighted sum $g(x, y) = \sum\sum w(s, t) f(x + s, y + t)$.
3. Correlation uses the kernel as written; convolution reverses it. Symmetric kernels give the same result.
4. Borders require a policy such as omission, padding, replication, or truncation.

Smoothing

5. Mean filtering averages a neighborhood; larger supports remove more fine detail.
6. Weighted smoothing emphasizes selected neighbors, typically giving the center more influence.
7. Smoothing can expose gross structures for later thresholding; median filtering can outperform averaging on some noise.

Sharpening

8. First differences detect change; second differences respond strongly to fine detail and produce a double response at gray-level steps.
9. The Laplacian is $\nabla^2 f = f_{xx} + f_{yy}$; with the source kernel, sharpening uses $g = f - \nabla^2 f$.
10. Unsharp masking subtracts a blur to form a high-frequency mask and adds that mask to the original.
11. The gradient magnitude is $\sqrt{G_x^2 + G_y^2}$ and is often approximated by $|G_x| + |G_y|$.
12. Sobel filtering uses two 3×3 derivative masks and is typically used for edge detection.

Combined enhancement

13. The lecture's bone-scan example combines Laplacian sharpening, Sobel edges, averaging, masking, addition, and a power-law transform.
14. Different operators can be chained because they emphasize complementary image information.

LECTURE 6

Edges and the Canny Detector

From image discontinuities to thin, linked edge curves.

Gradients · Noise · Derivative of Gaussian · Non-maximum suppression · Canny

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 23

Edge Detection Fundamentals

23.1 Purpose and Origin of Edges

Edge detection seeks locations of sudden change, or *discontinuity*, in an image. The lecture motivates edges as a compact representation: much of the semantic and shape information visible in a dense pixel array is concentrated around boundaries. An artist's line drawing is the idealized analogy, although an artist also uses object-level knowledge that a purely local edge operator does not possess.

Edges can arise for several different physical reasons. A boundary in the image may correspond to a **depth discontinuity**, a **surface-color discontinuity**, an **illumination discontinuity**, or a **surface-normal discontinuity**. Thus, a detected image edge is evidence of a visual change, not by itself a statement about which physical cause produced that change.

> **Edge as an intensity discontinuity**
>
> An edge is a location where the image-intensity function changes rapidly. Along a one-dimensional scanline, an ideal intensity step produces an extremum in the first derivative.

23.2 Image Gradient and Edge Strength

For a two-dimensional image $f(x, y)$, the gradient is

$$\nabla f(x, y) = \begin{bmatrix} \dfrac{\partial f}{\partial x} & \dfrac{\partial f}{\partial y} \end{bmatrix}. \tag{23.1}$$

where

- $f(x, y)$: image intensity at spatial location (x, y);
- $\partial f/\partial x$: rate of intensity change in the horizontal direction;
- $\partial f/\partial y$: rate of intensity change in the vertical direction;
- ∇f: vector pointing in the direction of most rapid intensity increase.

The gradient direction is

$$\theta = \operatorname{atan2}\left(\frac{\partial f}{\partial y}, \frac{\partial f}{\partial x}\right), \tag{23.2}$$

where

- θ: gradient orientation;
- $\partial f/\partial x$ and $\partial f/\partial y$: the two gradient components from eq. (23.1).

The two-argument arctangent preserves the quadrant and handles a zero horizontal component; orientation is undefined when both gradient components vanish. The lecture asks how this direction relates to the edge direction. The answer used later in edge linking is geometric: the **gradient is normal to the local edge**, so the edge tangent is perpendicular to the gradient.

Edge strength is represented by gradient magnitude:

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}. \tag{23.3}$$

where

- $\|\nabla f\|$: local edge-strength measure;
- the two squared terms: horizontal and vertical derivative energy.

23.3 Finite-Difference Filters

The spatial derivatives can be approximated with finite-difference filters. The lecture compares three standard pairs:

Table 23.1. Finite-difference masks shown in the edge lecture.

Operator	M_x	M_y
Prewitt	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$
Sobel	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$
Roberts	$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

The lecture's example images emphasize that the two directional masks respond to different orientation structure; together they provide the two components needed for gradient magnitude and orientation.

23.4 Noise and the Need to Smooth

Finite differences also react strongly to noise because an isolated noisy pixel can differ sharply from its neighbors. The larger the noise fluctuation, the larger the derivative response can become. Consequently, direct differentiation can make it difficult to decide which response belongs to a real edge.

The proposed remedy is to **smooth first**. For a one-dimensional illustration, edge detection can be expressed as searching for peaks of

$$\frac{d}{dx}(f * g), \tag{23.4}$$

where

- f: input signal or image row;
- g: smoothing kernel;
- $*$: convolution;
- d/dx: differentiation along the chosen coordinate.

Figure unavailable

`edge_smooth_first.jpg`

Figure 23.1. Source illustration of smoothing before differentiation: smoothing suppresses high-frequency noise so the derivative peak associated with the underlying edge is easier to identify.

23.5 Derivative Theorem of Convolution

Differentiation can itself be represented through convolution, and convolution is associative. The lecture therefore rewrites the smooth-then-differentiate operation as

$$\frac{d}{dx}(f * g) = f * \frac{dg}{dx}. \tag{23.5}$$

where

- f: input image or signal;
- g: smoothing function;
- dg/dx: derivative of the smoothing function;
- the equality: permits smoothing and differentiation to be implemented with one derivative-of-smoothing filter.

The lecture implements this idea with derivatives of a Gaussian. It explicitly raises separability as a filter-design question and shows x- and y-direction derivative-of-Gaussian filters for detecting changes in orthogonal directions.

23.6 Smoothing Scale and Localization

A smoothed derivative reduces noise, but smoothing also broadens the edge response. Increasing the Gaussian spread therefore reveals edges at progressively larger scales while degrading precise localization of fine detail. The lecture summarizes this as a **tradeoff between smoothing and localization**: small scales preserve fine features; larger scales suppress more noise and emphasize coarser structure.

> **Chapter summary.** Edges are rapid image-intensity changes. The image gradient supplies edge direction and strength, finite-difference masks approximate derivatives, and smoothing is needed because derivatives amplify noise. Convolving with a derivative of a Gaussian combines smoothing and differentiation, but its scale controls a tradeoff between noise suppression and localization.

CHAPTER 24

Localizing and Linking Edge Curves

24.1 Why Gradient Ridges Must Be Thinned

After smoothing and differentiation, the gradient magnitude is often large along a thick ridge rather than at a single pixel. Edge extraction must therefore identify the actual ridge center and then connect neighboring edge points into curves.

24.2 Non-Maximum Suppression

The lecture proposes **non-maximum suppression**: retain points where gradient magnitude is locally maximal along a slice normal to the edge curve. If the current point is q, compare its response with interpolated responses at locations p and r on opposite sides along the gradient direction. The point q is retained only when it exceeds both neighboring values.

> **Direction used by non-maximum suppression**
>
> The comparison is performed *along the gradient direction*, because this direction is normal to the edge. Suppression therefore removes the sides of a thick response ridge while keeping its local center.

The lecture emphasizes interpolation because p and r need not land exactly on integer pixel centers after the comparison direction is aligned with the local gradient.

24.3 Predicting the Next Edge Point

Once a retained point is assumed to lie on an edge, the local edge tangent is constructed perpendicular to the gradient. This tangent predicts which neighboring candidate point should continue the curve. Edge linking therefore combines a local response criterion with local geometric continuity.

24.4 Criteria for an Optimal Edge Detector

The lecture gives three criteria for designing an "optimal" edge detector:

1. **Good detection:** minimize both false positives from noise and false negatives that miss true edges.
2. **Good localization:** detected edge positions should lie as close as possible to the true edges.
3. **Single response:** produce only one detector response for each true edge point, minimizing multiple local maxima near the same edge.

These criteria explain why smoothing, localization, thinning, and linking cannot be considered independently.

Chapter summary. Gradient magnitude produces thick response ridges rather than immediately giving final edge curves. Non-maximum suppression keeps local maxima along the gradient normal, while the perpendicular tangent predicts edge continuation. A good detector balances detection, localization, and a single response per true edge.

CHAPTER 25

The Canny Edge Detector

25.1 Model and Motivation

The Canny detector is presented as one of the most widely used edge detectors in computer vision. Its theoretical model assumes **step edges corrupted by additive Gaussian noise**. Canny showed that the first derivative of a Gaussian closely approximates an operator that optimizes the product of signal-to-noise ratio and localization.

The source cites: J. Canny, "A Computational Approach To Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8:679–714, 1986.

25.2 Algorithm

The lecture gives the following pipeline:

1. filter the image with a derivative of Gaussian;
2. compute gradient magnitude and orientation;
3. perform non-maximum suppression to reduce multi-pixel-wide ridges to single-pixel width;
4. link and threshold edges with hysteresis.

Figure unavailable

`edge_canny_stages.jpg`

Figure 25.1. Source Canny example: original image, gradient magnitude, thresholded response, and thinned result after non-maximum suppression.

25.3 Hysteresis Thresholding

Canny linking uses two thresholds, **high** and **low**. The high threshold starts curves from strong edge evidence. The low threshold allows weaker responses to continue a curve when they are connected to strong responses. Weak isolated responses are therefore rejected even if they exceed the low threshold.

25.4 Effect of Gaussian Scale

The Gaussian spread σ determines the scale of the structures emphasized by the detector. The lecture's examples show that a **large σ detects large-scale edges**, whereas a **small σ detects fine features**. The

Figure unavailable

edge_hysteresis.jpg

Figure 25.2. Hysteresis thresholding in the lecture: strong edges obtained with the high threshold provide seeds, while connected weak edges from the low-threshold result can be retained.

appropriate value depends on the desired behavior.

Figure unavailable

edge_sigma_effect.jpg

Figure 25.3. Effect of Gaussian spread in Canny detection: increasing σ suppresses finer detail and retains coarser-scale edge structure.

25.5 Edges as an Intermediate Representation

The lecture closes by stressing that edge detection is only the beginning. An edge map still needs grouping and interpretation before it becomes an object or region description. The Berkeley segmentation example contrasts the original image, human segmentation, and gradient magnitude to illustrate the gap between local boundary evidence and complete scene segmentation. The source provides the Berkeley segmentation benchmark page http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/.

> **Chapter summary.** Canny combines derivative-of-Gaussian filtering, gradient computation, non-maximum suppression, and hysteresis linking. The two-threshold scheme distinguishes strong seeds from connected weak responses, while σ controls the scale of detected structure. The result remains an intermediate representation that subsequent grouping must interpret.

Lecture 6 Quick Review

Edge fundamentals

1. Edges are rapid image-intensity changes caused by depth, color, illumination, or surface-normal discontinuities.
2. ∇f points toward the most rapid intensity increase; its magnitude measures edge strength, and it is normal to the local edge.
3. Prewitt, Sobel, and Roberts masks approximate directional derivatives.
4. Derivatives amplify noise, so smoothing is performed before or together with differentiation.
5. $\frac{d}{dx}(f * g) = f * g'$ permits a derivative-of-Gaussian implementation.

Localization and Canny

6. Non-maximum suppression keeps maxima along the gradient direction and thins thick ridges.
7. A good detector aims for good detection, good localization, and a single response.
8. Canny performs derivative-of-Gaussian filtering, gradient estimation, non-maximum suppression, and hysteresis linking.
9. High hysteresis threshold starts curves; the low threshold continues connected weaker responses.
10. Large σ emphasizes coarse edges; small σ preserves finer features.

LECTURE 7

Corners, Blobs, and Invariant Features

From repeatable interest points to scale- and affine-adapted regions.

Harris corners · Second moment matrix · Scale space · LoG/DoG · Affine adaptation

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 26

Feature Extraction and Harris Corners

26.1 Why Extract Features?

The lecture motivates features through panorama stitching. Given two overlapping images, the basic sequence is: **extract features**, **match features**, and **align the images**. Feature extraction replaces a dense pixel representation with a smaller collection of distinctive local structures that can be recovered in multiple views.

Good features should satisfy four properties:

- **repeatability**: the same feature can be found despite geometric and photometric transformations;
- **saliency**: each feature should have a distinctive description;
- **compactness and efficiency**: there should be far fewer features than pixels;
- **locality**: a feature occupies a relatively small image region and is therefore more robust to clutter and occlusion.

Feature points support motion tracking, image alignment, 3D reconstruction, object recognition, indexing and database retrieval, and robot navigation.

26.2 Corner Intuition

A Harris corner is motivated by the local effect of shifting a small image window. In a **flat region**, shifting the window causes little change in every direction. Along an **edge**, there is little change for shifts parallel to the edge but substantial change across it. Around a **corner**, shifts in essentially every direction produce a significant intensity change. Equivalently, a corner contains two or more dominant gradient directions.

The source attributes this detector to C. Harris and M. Stephens, "A Combined Corner and Edge Detector," *Proceedings of the 4th Alvey Vision Conference*, pp. 147–151.

26.3 PCA View of Local Gradient Distributions

The lecture introduces principal component analysis (PCA) to interpret dominant gradient directions. A scalar λ is an eigenvalue of an $n \times n$ matrix A if a nonzero vector x satisfies

$$Ax = \lambda x. \tag{26.1}$$

where

- A: square matrix;
- x: eigenvector;
- λ: corresponding eigenvalue.

The PCA recipe in the lecture is: subtract the mean, compute the covariance matrix, compute its eigenvectors and eigenvalues, and rank components by eigenvalue. The first component is the direction of highest

variance; subsequent components maximize variance subject to orthogonality to the earlier components.

26.4 Harris Change Function

For a displacement (u, v), Harris measures how much a window changes:

$$E(u,v) = \sum_{x,y} w(x,y)\,[I(x+u, y+v) - I(x,y)]^2\,. \tag{26.2}$$

where

- $I(x, y)$: image intensity;
- (u, v): small window displacement;
- $w(x, y)$: window function, either a hard window or a Gaussian weighting;
- $E(u, v)$: weighted squared intensity change caused by the shift.

The slide first presents a second-order expansion about $(0, 0)$:

$$\begin{aligned} E(u,v) \approx E(0,0) &+ \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} E_u(0,0) \\ E_v(0,0) \end{bmatrix} \\ &+ \frac{1}{2} \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} E_{uu}(0,0) & E_{uv}(0,0) \\ E_{uv}(0,0) & E_{vv}(0,0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}. \end{aligned} \tag{26.3}$$

where

- E_u, E_v: first partial derivatives of E with respect to displacement;
- E_{uu}, E_{uv}, E_{vv}: second partial derivatives evaluated at zero shift;
- (u, v): the small displacement being evaluated.

For small shifts, the lecture simplifies the result to the quadratic form

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}, \tag{26.4}$$

where the **second moment matrix** is

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}. \tag{26.5}$$

where

- I_x, I_y: local image derivatives;
- $w(x, y)$: spatial weighting window;
- M: symmetric 2×2 second moment matrix describing local gradient variation.

26.5 Interpreting the Second Moment Matrix

For an axis-aligned case,

$$M = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}.$$

If either eigenvalue is close to zero, the region does not behave like a corner. A corner requires both eigenvalues to be large.

In the general case, because M is symmetric, the lecture writes

$$M = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R, \tag{26.6}$$

where

- λ_1, λ_2: eigenvalues controlling the ellipse axes;
- R: orientation transform determined by the eigenvectors.

The local change can be visualized through the ellipse

$$\begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix} = \text{const.} \tag{26.7}$$

The ellipse orientation is determined by the eigenvectors, and its axis lengths vary inversely with the square roots of the eigenvalues. Large eigenvalues mean strong change in the associated directions.

The lecture's classification is therefore:

- both λ_1 and λ_2 small $\rightarrow$ flat region;
- one large and one small $\rightarrow$ edge;
- both large and of comparable scale $\rightarrow$ corner.

26.6 Harris Corner Response and Algorithm

Instead of explicitly thresholding two eigenvalues, Harris uses

$$R = \det(M) - \alpha \ \text{trace}(M)^2 = \lambda_1\lambda_2 - \alpha(\lambda_1 + \lambda_2)^2, \tag{26.8}$$

where

- R: Harris corner response;
- $\det(M) = \lambda_1\lambda_2$;
- $\text{trace}(M) = \lambda_1 + \lambda_2$;
- α: constant in the range 0.04–0.06 in the lecture.

Corners give positive, large R; edge configurations produce negative R; flat regions have small $|R|$.

The complete detector is summarized in five steps:

1. compute Gaussian derivatives at each pixel;
2. compute M in a Gaussian window around each pixel;
3. compute the corner response R;
4. threshold R;
5. retain local maxima of R using non-maximum suppression.

26.7 Invariance Properties

The lecture evaluates Harris features under common image transformations. Rotation changes the orientation of the second-moment ellipse but preserves its eigenvalues, so the response is rotation invariant. Because

Figure unavailable

harris_results.jpg

Figure 26.1. Harris detector result from the lecture after response computation, thresholding, and non-maximum suppression.

only derivatives are used, adding an intensity offset $I \mapsto I + b$ does not change the derivatives. Intensity scaling $I \mapsto aI$ changes the response magnitude, so invariance to a general affine intensity change is only partial. Most importantly, the ordinary Harris detector is **not scale invariant**: a corner at one scale can be classified as an edge after rescaling.

> **Chapter summary.** Good local features should be repeatable, salient, compact, and local. Harris corners measure the intensity change produced by small window shifts. The second moment matrix summarizes local gradient variation; its eigenvalues distinguish flat regions, edges, and corners. The Harris response provides a practical scalar test followed by thresholding and non-maximum suppression, but standard Harris detection is not scale invariant.

CHAPTER 27

Scale-Invariant Blob Detection

27.1 From Edges to Blobs

The lecture connects first- and second-derivative reasoning. A first derivative of a smoothed step has a maximum at an edge; a second derivative crosses zero at the edge. A blob can be viewed as the superposition of two edge-like ripples. The magnitude of a Laplacian response reaches a maximum near the blob center when the filter scale matches the blob scale.

27.2 Why Scale Normalization Is Needed

If an image is filtered with Laplacians at progressively larger scales, the raw derivative response decays as σ increases. For the perfect step-edge example, the lecture shows the first-derivative Gaussian peak decreasing as

$$A(\sigma) = \frac{1}{\sigma\sqrt{2\pi}}. \tag{27.1}$$

where

- $A(\sigma)$: peak first-derivative response in the source step-edge example;
- σ: Gaussian scale;
- the inverse dependence on σ: larger smoothing scales reduce the unnormalized derivative amplitude.

To keep a first-derivative response comparable across scale, the lecture therefore multiplies the Gaussian derivative by σ. Since the Laplacian is second order, it uses σ^2 for scale normalization.

The two-dimensional Laplacian of a Gaussian g is

$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}, \tag{27.2}$$

and the scale-normalized form in the lecture is

$$\nabla^2_{\text{norm}} g = \sigma^2 \left(\frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \right). \tag{27.3}$$

where

- g: Gaussian smoothing function;
- σ: Gaussian scale;
- $\nabla^2 g$: Laplacian response;
- σ^2: normalization factor used to compare second-derivative responses across scales.

The source also displays the radial Laplacian shape proportionally as

$$(x^2 + y^2 - 2\sigma^2) e^{-(x^2+y^2)/(2\sigma^2)}. \tag{27.4}$$

where

- (x, y): offset from the filter center;
- σ: Gaussian scale;
- the expression: source-provided radial shape used to explain scale selection.

27.3 Characteristic Scale

For a binary circle of radius r, the lecture states that the magnitude of the scale-normalized Laplacian response at the blob center reaches its maximum at

$$\sigma = \frac{r}{\sqrt{2}}. \tag{27.5}$$

where

- r: radius of the binary circular blob;
- σ: filter scale producing the peak Laplacian response.

The **characteristic scale** of a region is defined as the scale that produces this peak response.

27.4 Scale-Space Blob Detector

The lecture's scale-space detector is:

1. convolve the image with scale-normalized Laplacians at several scales;
2. find maxima of the **squared Laplacian response** in scale-space.

The source cites T. Lindeberg, "Feature detection with automatic scale selection," *International Journal of Computer Vision*, 30(2):77–116, 1998.

Figure unavailable

`blob_scale_space_example.jpg`

Figure 27.1. Scale-space blob detections from the lecture. Circle size visualizes the selected characteristic scale.

27.5 Difference of Gaussians

For efficient implementation, the lecture approximates the scale-normalized Laplacian using a Difference of Gaussians (DoG):

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right), \tag{27.6}$$

$$\mathrm{DoG} = G(x, y, k\sigma) - G(x, y, \sigma). \tag{27.7}$$

where

- $G(x, y, \sigma)$: Gaussian at scale σ;
- G_{xx}, G_{yy}: second spatial derivatives;
- L: scale-normalized Laplacian quantity used in the comparison;
- k: multiplicative separation between the two Gaussian scales;
- DoG: difference of the Gaussian kernels; convolving it with the image gives the difference between the two smoothed images.

For $k > 1$ close to 1, $\mathrm{DoG} \approx (k-1)\sigma^2 \nabla^2 G$; the factor $k-1$ makes the proportionality explicit. The lecture cites D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *IJCV*, 60(2):91–110, 2004.

> **Chapter summary.** Scale selection turns blob detection into a search across both position and scale. Raw derivative amplitudes decay as scale increases, so the second-order Laplacian is normalized by σ^2. A blob's characteristic scale is the scale of maximum response; for a binary circle, $\sigma = r/\sqrt{2}$. Difference of Gaussians provides an efficient approximation used in scale-invariant feature pipelines.

CHAPTER 28

Affine Adaptation and Invariant Description

28.1 From Scale Invariance to Affine Invariance

Scale invariance handles isotropic resizing, but viewpoint changes can deform a circular region into an ellipse. The lecture reuses the Harris second moment matrix to describe this local shape. The resulting second-moment ellipse can be interpreted as the region's **characteristic shape**.

Affine adaptation normalizes the elliptical region by transforming it to a unit circle. This removes local affine shape distortion before an appearance descriptor is computed.

28.2 Orientation Ambiguity

There is no unique transformation from an ellipse to a unit circle: the normalized circle can still be rotated or flipped without changing its shape. To obtain a repeatable orientation, the lecture proposes:

1. create a histogram of local gradient directions inside the patch;
2. assign the canonical orientation at the peak of a smoothed orientation histogram.

This step removes the rotational ambiguity after affine shape normalization.

28.3 Iterative Affine Adaptation

A practical problem remains: the window $w(x, y)$ used to compute the second moment matrix should itself match the characteristic region shape. The lecture therefore uses an iterative strategy:

1. begin with a circular window and compute the second moment matrix;
2. infer an ellipse-shaped characteristic region and normalize it;
3. recompute the second moment matrix using the updated window;
4. repeat until the adapted region stabilizes.

The source cites K. Mikolajczyk and C. Schmid, "Scale and Affine Invariant Interest Point Detectors," *IJCV*, 60(1):63–86, 2004, and provides the project page `http://www.robots.ox.ac.uk/~vgg/research/affine/`.

Figure unavailable

`blob_affine_adaptation.jpg`

Figure 28.1. Affine-adapted regions from the lecture: scale-invariant circular detections become ellipses that follow local affine deformation.

28.4 Feature-Extraction Pipeline

The lecture summarizes a complete invariant-feature pipeline as: extract affine regions, normalize each region, eliminate rotational ambiguity, and compute an appearance descriptor such as SIFT.

Figure unavailable

`feature_extraction_summary.jpg`

Figure 28.2. Lecture summary of feature extraction: affine-region detection, normalization, orientation assignment, and appearance description.

28.5 Invariance and Covariance

The lecture makes an important distinction:

$$\text{features}(\text{transform}(\text{image})) = \text{features}(\text{image}), \tag{28.1}$$

$$\text{features}(\text{transform}(\text{image})) = \text{transform}(\text{features}(\text{image})). \tag{28.2}$$

where

- eq. (28.1): invariance – the representation itself is unchanged;
- eq. (28.2): covariance – detected features move or deform consistently with the image transformation.

The lecture's conclusion is that **covariant detection enables invariant description**: first detect regions that transform predictably, then normalize them so the descriptor can remain stable.

> **Chapter summary.** Affine adaptation uses the second-moment ellipse as a characteristic local shape, repeatedly normalizes that shape, and assigns a canonical orientation from local gradient directions. The result is a covariantly detected region that can be normalized into an invariant descriptor frame.

Lecture 7 Quick Review

Corners

1. Good features are repeatable, salient, compact, efficient, and local.
2. Harris corners produce large intensity change for shifts in every direction.
3. $M = \sum w\,[I_x, I_y]^T [I_x, I_y]$ is the second moment matrix.
4. Two small eigenvalues indicate a flat region; one large eigenvalue indicates an edge; two large eigenvalues indicate a corner.
5. $R = \det M - \alpha\,\mathrm{trace}(M)^2$ provides a scalar corner response.
6. Standard Harris detection is rotation invariant but not scale invariant.

Scale and affine adaptation

7. Scale-normalized blob detection uses $\sigma^2\nabla^2 G$ and searches for extrema over space and scale.
8. A binary circular blob of radius r peaks at $\sigma = r/\sqrt{2}$ in the lecture model.
9. DoG approximates the Laplacian response efficiently.
10. The second-moment ellipse supplies a characteristic local shape for affine adaptation.
11. Canonical orientation is assigned from a local gradient-orientation histogram.
12. Covariant detection followed by normalization supports invariant description.

LECTURE 8

Fitting

From votes and least squares to robust estimation and model selection.

Hough transform · Least squares · Robust estimators · RANSAC · K-lines

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 29

Model Fitting and the Hough Transform

29.1 What Fitting Tries to Solve

Fitting chooses a **parametric model** to represent a set of measured features. The model may be simple, such as a line or circle, or much more complex, such as the outline or configuration of a car. Membership is not purely local: a feature point cannot generally be assigned to a model instance by inspecting that point alone.

The lecture organizes fitting around three questions:

1. What model best represents a given set of features?
2. If several model instances are present, which feature belongs to which instance?
3. How many model instances are present?

Practical fitting must handle noisy feature locations, clutter and other outliers, multiple structures, and missing data due to occlusion. Computational complexity matters because exhaustive search over every possible parameter combination and feature assignment is infeasible.

29.2 Voting Schemes

A voting method lets each feature vote for all models compatible with that feature. Noise features should not vote consistently for the same model, and missing data are tolerable as long as enough correct features remain to support a strong model hypothesis.

The **Hough transform** is an early voting scheme:

1. discretize model-parameter space into bins;
2. for every image feature, vote for every parameter bin that could have produced that feature;
3. detect model hypotheses as peaks in the accumulator.

29.3 Line Parameterization

For the slope-intercept model $y = mx + b$, one image point (x_0, y_0) is compatible with every pair (m, b) satisfying

$$b = -x_0 m + y_0. \tag{29.1}$$

where

- (x_0, y_0): fixed point in image space;
- (m, b): slope and intercept coordinates in Hough parameter space.

Thus, an image point maps to a line in (m, b) space, and two image points determine an image line at the intersection of their two Hough-space lines.

The slope-intercept representation is problematic because its parameter domain is unbounded and vertical

lines require infinite slope. The lecture therefore uses the polar line representation

$$x \cos\theta + y \sin\theta = \rho. \tag{29.2}$$

where

- (x, y): image point;
- θ: orientation of the line normal;
- ρ: signed distance of the line from the origin along that normal.

Each image point traces a sinusoid in (θ, ρ) parameter space. A set of collinear points produces sinusoids that intersect near the same accumulator location.

29.4 Hough Algorithm for Lines

The lecture's algorithm is:

1. initialize accumulator H to zero;
2. for every edge point (x, y) and every candidate θ from 0 to 180 degrees, compute $\rho = x \cos\theta + y \sin\theta$ and increment $H(\theta, \rho)$;
3. find local maxima of H;
4. convert each peak (θ, ρ) back to the image line in eq. (29.2).

Figure unavailable

`hough_basic.jpg`

Figure 29.1. Basic Hough example from the lecture: image features generate parameter-space votes, and a concentrated peak identifies a common line.

The source also illustrates circles, several simultaneous lines, fuzzy peaks caused by positional noise, and spurious peaks from random points.

29.5 Practical Details, Strengths, and Limitations

Several implementation choices strongly affect Hough detection:

- reject irrelevant features when possible, for example by retaining edge points with significant gradient magnitude;
- choose accumulator resolution carefully: too coarse merges different models, while too fine divides the votes of nearly collinear noisy points;
- optionally increment neighboring bins, effectively smoothing the accumulator;
- tag votes when later assignment of features to detected model instances is needed.

The Hough transform processes points independently, tolerates occlusion and some noise, handles non-local structure, and can detect multiple model instances in one pass. Its main disadvantages are the exponential growth of search complexity with the number of model parameters, spurious peaks produced by non-target structure, and the difficulty of selecting an appropriate accumulator grid.

Chapter summary. Fitting asks which model best explains measured features, how features should be assigned to instances, and how many instances exist. Hough voting converts local evidence into peaks in a discretized parameter space. Polar line coordinates avoid the vertical-line problem of slope-intercept form, but accumulator resolution and parameter-space dimensionality remain important practical limitations.

CHAPTER 30

Least Squares and Robust Fitting

30.1 Vertical Least-Squares Line Fitting

For data (x_i, y_i) and line $y_i = mx_i + b$, ordinary least squares minimizes

$$E = \sum_i (y_i - mx_i - b)^2 = \|Y - XB\|^2, \tag{30.1}$$

with

$$Y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}, \qquad X = \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} m \\ b \end{bmatrix}.$$

Expanding the squared norm as done in the lecture gives

$$E = (Y - XB)^T (Y - XB) = Y^T Y - 2(XB)^T Y + (XB)^T (XB). \tag{30.2}$$

Differentiating with respect to B yields

$$\frac{dE}{dB} = 2X^T XB - 2X^T Y = 0, \tag{30.3}$$

and therefore the normal equations

$$X^T XB = X^T Y. \tag{30.4}$$

where

- (x_i, y_i): measured data points;
- m, b: line slope and intercept;
- X, Y, B: matrix form of the observations and unknown parameters;
- eq. (30.4): normal equations for the least-squares solution to $XB = Y$.

The lecture warns that this "vertical" least-squares formulation is not rotation invariant and fails for vertical lines because its residual measures only vertical displacement.

30.2 Total Least Squares

A rotation-symmetric line model is

$$ax + by = d, \qquad a^2 + b^2 = 1, \tag{30.5}$$

where $N = (a, b)$ is a unit normal. The perpendicular distance of point (x_i, y_i) to the line has magnitude $|ax_i + by_i - d|$, so total least squares minimizes

$$E = \sum_i (ax_i + by_i - d)^2. \tag{30.6}$$

where

- (a, b): unit line normal;
- d: signed line offset from the origin;
- E: sum of squared perpendicular distances.

Differentiating with respect to d gives the source stationarity condition

$$\frac{\partial E}{\partial d} = \sum_i -2\big(ax_i + by_i - d\big) = 0, \tag{30.7}$$

which yields the lecture's centroid relation

$$d = a\bar{x} + b\bar{y}. \tag{30.8}$$

Source notation discrepancy

In the intermediate summation printed on the source slide, the second summand is written with x_i even though the same line concludes with $a\bar{x} + b\bar{y}$. The final centroid expression and the surrounding geometry make the intended second summation over y_i unambiguous, so eq. (30.8) uses that intended form.

Center the observations in

$$U = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix}.$$

The corresponding second moment matrix is

$$U^T U = \begin{bmatrix} \sum_i (x_i - \bar{x})^2 & \sum_i (x_i - \bar{x})(y_i - \bar{y}) \\ \sum_i (x_i - \bar{x})(y_i - \bar{y}) & \sum_i (y_i - \bar{y})^2 \end{bmatrix}. \tag{30.9}$$

Then

$$E = (UN)^T (UN), \qquad \|N\|^2 = 1. \tag{30.10}$$

To impose the unit-norm constraint, introduce a Lagrange multiplier λ:

$$\nabla_N \left[N^T U^T U N - \lambda (N^T N - 1) \right] = 0 \quad \Longrightarrow \quad U^T U N = \lambda N. \tag{30.11}$$

The minimizing unit normal is the eigenvector of $U^T U$ associated with its smallest eigenvalue; that eigenvalue is the minimum squared residual.

Constrained-minimization correction

The source writes $2(U^T U)N = 0$, which is the unconstrained stationarity equation. It generally has no unit-norm solution for noisy data. The Lagrange-multiplier condition in eq. (30.11) gives the correct constrained eigenproblem.

where

- $(\bar{x}, \bar{y})$: sample centroid;
- U: matrix of centered point coordinates;
- $N = (a, b)^T$: unit normal;
- $U^T U$: second moment matrix of the centered point cloud.

The lecture also presents least squares probabilistically through the generative model

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix} + \varepsilon \begin{bmatrix} a \\ b \end{bmatrix}, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2). \tag{30.12}$$

where

- (u, v): an ideal point on the line $au + bv = d$;
- (x, y): observed noisy point;
- (a, b): unit normal direction to the line;
- ε: zero-mean Gaussian displacement along the normal direction;
- σ: standard deviation of that perpendicular noise.

This connects the perpendicular-distance objective to likelihood maximization under the lecture's Gaussian generative model.

30.3 General Curves and Conics

For a general implicit curve $C(u, v) = 0$, geometric least squares would minimize squared distance from each data point to its nearest point on the curve. At the closest point (u_0, v_0), the connection to the observation (x, y) is normal to the curve, so the source writes the system

$$C(u_0, v_0) = 0, \tag{30.13}$$

$$\frac{\partial C}{\partial v}(u_0, v_0)(x - u_0) - \frac{\partial C}{\partial u}(u_0, v_0)(y - v_0) = 0. \tag{30.14}$$

where

- $C(u, v) = 0$: implicit curve;
- (x, y): measured point;
- (u_0, v_0): closest point on the curve;
- the second relation: orthogonality condition from the lecture.

> **Source equation typo**
>
> The displayed source equation repeats $\partial C/\partial v$ in both terms, while the tangent vector printed immediately above it is $(\partial C/\partial v, -\partial C/\partial u)$. The second term in eq. (30.14) uses $\partial C/\partial u$, which is the unambiguous correction implied by that tangent vector and the stated orthogonality condition.

Solving for the true geometric distance is nonlinear even for a conic.

A general conic is written

$$C(a, x) = a \cdot x = ax^2 + bxy + cy^2 + dx + ey + f = 0, \tag{30.15}$$

with parameter vector $a = [a, b, c, d, e, f]^T$ and monomial vector $x = [x^2, xy, y^2, x, y, 1]^T$. For observa-

tions (x_i, y_i), the algebraic constraints can be stacked into the source's homogeneous linear system

$$\begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^2 & x_2y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n^2 & x_ny_n & y_n^2 & x_n & y_n & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix} = 0. \tag{30.16}$$

where

- (x_i, y_i): observed points;
- a, b, c, d, e, f: conic coefficients;
- each row: monomials evaluated at one observation;
- the zero right-hand side: the algebraic curve constraint $C(a, x_i) = 0$.

The lecture contrasts nonlinear geometric-distance fitting with least squares on the **algebraic distance** $C(a, x)$. The homogeneous system requires a nonzero coefficient vector: for noisy data, minimize the squared algebraic residual subject to a normalization such as $\|a\| = 1$, using the right singular vector associated with the smallest singular value. Without a normalization, the all-zero coefficients give a trivial solution and do not define a conic.

30.4 Why Ordinary Least Squares Is Not Robust

A squared residual penalizes a large outlier extremely strongly, so even one distant point can distort a least-squares fit. Robust estimation replaces squared error with a saturating penalty:

$$\min_{\theta} \sum_i \rho\big(r_i(x_i, \theta); \sigma\big), \tag{30.17}$$

where the source example uses

$$\rho(u; \sigma) = \frac{u^2}{\sigma^2 + u^2}. \tag{30.18}$$

where

- $r_i(x_i, \theta)$: residual of observation i under model parameters θ;
- ρ: robust loss;
- σ: scale controlling the transition from approximately quadratic behavior to saturation;
- u: residual value passed to the robust function.

Figure unavailable

`robust_loss_source.jpg`

Figure 30.1. Source robust-loss illustration. Small residuals are treated similarly to squared error, whereas large residuals saturate and therefore have limited influence.

The scale σ must be chosen carefully. If it is too small, many points are effectively treated as outliers and the fit can become poor. If it is too large, the method behaves much like ordinary least squares. Robust fitting is nonlinear and is generally solved iteratively; least squares can provide initialization, and the lecture mentions an adaptive scale based on a "magic number" times the median residual.

Chapter summary. Ordinary least squares is simple but orientation dependent in slope-intercept form and highly sensitive to outliers. Total least squares minimizes perpendicular distances and reduces to an eigenproblem on the centered second-moment matrix. For general curves, geometric distance is nonlinear. Robust losses limit the influence of large residuals but introduce an iterative optimization problem and a scale parameter that must be selected appropriately.

CHAPTER 31

RANSAC and Multiple-Model Fitting

31.1 Random Sample Consensus

When outlier rates become too large for a smooth robust loss, the lecture introduces Random Sample Consensus (RANSAC). Its general procedure is:

1. choose a small subset of observations uniformly at random;
2. fit a model to that subset;
3. classify the remaining observations as inliers or outliers according to their distance from the model;
4. repeat many times and retain the best supported model.

For line fitting, each trial draws s points, fits a line, finds points whose distance is below threshold t, and, if the consensus contains at least d inliers, refits using all inliers.

Figure unavailable

`ransac_line_sequence.jpg`

Figure 31.1. RANSAC line-fitting sequence from the lecture: random seed, provisional model, inlier identification, and refinement from the consensus set.

31.2 Choosing RANSAC Parameters

The lecture recommends choosing the initial sample size s as the minimum number of points required to determine the model. For a distance threshold t, it gives the Gaussian-noise relation

$$t^2 = 3.84\,\sigma^2 \tag{31.1}$$

for an inlier probability such as 0.95 for a single scalar residual (such as perpendicular line distance) under zero-mean Gaussian noise with standard deviation σ. A multidimensional residual requires the appropriate chi-squared quantile for its dimension.

where

- t: inlier-distance threshold;
- σ: standard deviation of the assumed measurement noise;
- 3.84: source-provided squared factor for the stated probability example.

If e is the outlier ratio and each random sample contains s points, the probability that one sample contains

no outliers is $(1-e)^s$. To obtain at least one outlier-free sample with desired probability p, the lecture uses

$$N = \left\lceil \frac{\log(1-p)}{\log(1-(1-e)^s)} \right\rceil. \tag{31.2}$$

where

- N: required number of random samples;
- p: desired probability that at least one sample is outlier free, for example 0.99;
- e: outlier ratio;
- s: points per minimal sample.

For $0 < e < 1$ and $0 < p < 1$, round up to ensure the requested confidence. This model assumes independent trials and inlier draws; degenerate samples must also be rejected. If $e = 0$, one nondegenerate sample suffices; if $e = 1$, no finite number can succeed. The consensus size d should be compatible with the expected inlier ratio. For the source example with $p = 0.99$, the required sample counts are: The table

Table 31.1. RANSAC iteration counts N reported in the lecture for $p = 0.99$.

$s \backslash e$	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

makes the exponential sensitivity to both minimal-sample size and outlier ratio explicit.

If e is unknown, the lecture proposes an adaptive procedure: start from a pessimistic assumption, measure the inlier fraction of each candidate model, update $e = 1 - (\text{inliers}/\text{total points})$, recompute N with eq. (31.2), and continue until the current sample count reaches the updated requirement.

31.3 RANSAC Strengths and Limitations

RANSAC is simple, general, and applicable to many problems. However, it contains several tunable parameters, minimal samples can sometimes provide poor initialization, many iterations may be needed, and extremely small inlier ratios can cause failure. The lecture therefore motivates alternatives that exploit additional structure instead of brute-force random sampling.

31.4 Incremental Line Fitting

For an ordered edge chain, incremental fitting exploits locality:

1. fit a line to the first s points;
2. keep adding sequential points while the fitting residual remains below a threshold, refitting as the group grows;
3. when the residual becomes too large, terminate the current line and begin a new one from the next unassigned points.

This method adapts the number of lines and exploits local ordering, but it requires a sequential feature order, cannot handle occlusion well, and is sensitive to noise and the residual threshold.

Figure unavailable

incremental_line_fitting.jpg

Figure 31.2. Incremental line fitting from the lecture. A line grows along an ordered edge chain until the residual threshold is violated, after which a new segment begins.

31.5 K-Lines

K-lines is an alternating-minimization procedure analogous to clustering:

1. initialize k lines, either by random parameters or by randomly partitioning points and fitting each group;
2. assign every point to its nearest line;
3. refit each line from its assigned points;
4. repeat until convergence.

Figure unavailable

klines_sequence.jpg

Figure 31.3. One K-lines iteration from the lecture: initialization, nearest-line assignment, and line refitting.

With exact nearest-line assignment and least-squares refitting of nonempty groups, the sum of squared perpendicular residuals cannot increase at an iteration and the method can cope with occlusion. Its disadvantages are that k must be known, convergence can occur at a local minimum, and results are sensitive to initialization.

> **Chapter summary.** RANSAC handles severe outlier contamination by repeatedly fitting minimal random samples and selecting the model with the strongest consensus. Its required sample count depends sharply on outlier ratio and minimal sample size. Multiple structures can also be fit using sequential locality through incremental fitting or alternating assignment/refitting through K-lines, each with different assumptions and failure modes.

CHAPTER 32

Model Complexity and Selection

32.1 Bias–Variance Tradeoff

The lecture closes the fitting material with the bias–variance tradeoff. Models with too many parameters can fit a particular sample very well but have high variance; their generalization error is caused by **overfitting**. Models with too few parameters cannot represent the data sufficiently well and have high bias; their generalization error is caused by **underfitting**.

32.2 Occam's Razor

If several models describe the observed data equally well, the lecture recommends preferring the simpler model. This is presented as a powerful rule of thumb rather than a fully rigorous model-selection rule: model error must be balanced against model complexity, and simpler models are often preferred because of their lower variance and greater robustness.

> **Chapter summary.** Model fitting is not only an estimation problem but also a model-selection problem. Increasing complexity reduces training error but can increase variance and overfitting; insufficient complexity causes bias and underfitting. Occam's razor favors the simpler explanation when competing models describe the observations equally well.

Lecture 8 Quick Review

Voting and least squares

1. Hough voting maps each feature to all compatible model parameters and detects models as accumulator peaks.
2. Polar line coordinates $x \cos\theta + y \sin\theta = \rho$ avoid infinite slopes.
3. Ordinary line least squares minimizes vertical residuals and satisfies $X^T XB = X^T Y$.
4. Total least squares minimizes perpendicular distance; its line normal is the smallest-eigenvalue eigenvector of $U^T U$.
5. General curve fitting by true geometric distance is nonlinear; algebraic conic fitting gives a linear alternative.

Robustness and multiple models

6. Saturating robust losses reduce the influence of large outliers but require an appropriate scale.
7. RANSAC repeatedly fits minimal samples, measures consensus, and refits from inliers.
8. $N = \lceil \log(1-p)/\log(1-(1-e)^s) \rceil$ relates confidence, outlier ratio, minimal sample size, and trial count.
9. Incremental fitting exploits ordered locality; K-lines alternates assignment and refitting for a known k.
10. Excessive model complexity causes high variance/overfitting; insufficient complexity causes high bias/underfitting.
11. Occam's razor prefers a simpler model when alternatives explain the observations equally well.

LECTURE 9

Image Alignment

From local correspondences to robust geometric registration and panoramas.

Transformations · Descriptors · Matching · RANSAC/Hough · Homographies · Blending

Computer Vision — Marco Raoul Marini

Notes prepared by Matt Clifford Trinidad Ramos

CHAPTER 33

Image Alignment and 2D Transformations

33.1 Motivation and Challenges

Image alignment is required in applications such as panorama stitching and recognition of object instances. The lecture highlights small image overlap, occlusion, and clutter as central challenges because only part of the scene may provide usable correspondence.

Two broad approaches are distinguished:

- **direct (pixel-based) alignment**: search for a transformation under which most pixels agree;
- **feature-based alignment**: search for a transformation under which extracted feature correspondences agree; a final pixel-based check may then verify the result.

33.2 Alignment as a Fitting Problem

Fitting in one image searches for a model that minimizes residuals of features x_i. Alignment instead searches for a transformation T between matched features from two images:

$$\min_T \sum_i \text{residual}\big(T(x_i), x_i'\big). \tag{33.1}$$

where

- x_i: feature in the first image;
- x_i': corresponding feature in the second image;
- T: geometric transformation between the two images;
- residual: disagreement measure after transformation.

The feature-based outline is: extract features, compute putative matches, hypothesize a transformation from a small compatible group, and verify it by searching for additional matches consistent with the same transformation.

33.3 Transformation Families

The lecture distinguishes similarity transformations (translation, scale, rotation), affine transformations, and projective transformations (homographies). Affine transformations are treated first because they are linear to fit, approximate viewpoint change for roughly planar objects under roughly orthographic cameras, and can initialize more complex models.

The basic transformations shown in the lecture are

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad \text{(scale)}, \tag{33.2}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\Theta & -\sin\Theta \\ \sin\Theta & \cos\Theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad \text{(rotation)}, \tag{33.3}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & \alpha_x \\ \alpha_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad \text{(shear)}, \tag{33.4}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad \text{(translation)}, \tag{33.5}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad \text{(affine)}. \tag{33.6}$$

where

- (x, y) and (x', y'): corresponding coordinates before and after transformation;
- s_x, s_y: scale factors;
- Θ: rotation angle;
- α_x, α_y: shear parameters;
- t_x, t_y: translation parameters;
- a, b, c, d, e, f: six free affine parameters.

The lecture summarizes an affine transform as any combination of translation, scale, rotation, and shear. Its reference table places the main model families in a hierarchy of increasing freedom: Here I is the identity,

Table 33.1. Two-dimensional transformation families summarized in the alignment lecture.

Name	Matrix form	D.O.F.	Preserves
Translation	$[\, I \mid t \,]_{2\times3}$	2	orientation and the stronger properties implied by translation
Rigid (Euclidean)	$[\, R \mid t \,]_{2\times3}$	3	lengths (and hence angles)
Similarity	$[\, sR \mid t \,]_{2\times3}$	4	angles
Affine	$[\, A \,]_{2\times3}$	6	parallelism
Projective	$[\, \widetilde{H} \,]_{3\times3}$	8	straight lines

R a rotation, s a uniform scale, t a translation, A an affine map, and $\widetilde{H}$ a projective homography. The source diagram emphasizes the nesting of these transformation classes.

33.4 Fitting an Affine Transformation

If correspondences are known, one match provides two equations:

$$\begin{bmatrix} x_i' \\ y_i' \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}. \tag{33.7}$$

Equivalently,

$$\begin{bmatrix} x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} x_i' \\ y_i' \end{bmatrix}. \tag{33.8}$$

where

- $(x_i, y_i) \leftrightarrow (x_i', y_i')$: one point correspondence;
- $m_1, \ldots, m_4, t_1, t_2$: six affine unknowns;
- each correspondence contributes two linearly independent equations.

At least three correspondences with non-collinear source points are required for a unique six-unknown affine fit. Collinear source points leave the system rank deficient. A nonsingular affine map also requires non-collinear destination points. With more data and a full-rank design matrix, least squares can be used.

> **Chapter summary.** Alignment can be direct or feature based, but feature-based alignment naturally becomes a fitting problem between paired observations. Similarity, affine, and projective models offer increasing flexibility. An affine transformation has six unknowns, each correspondence supplies two equations, and three matches with non-collinear source points are the minimum required for a unique linear solution.

CHAPTER 34

Feature Descriptors and Matching

34.1 Patch Descriptors

When correspondences are unknown, local image patches around interest points must be described and compared. Assuming geometric normalization has already removed the local transformation, the descriptor should be insensitive to intensity variation, noise, and perceptually insignificant pixel changes.

The simplest descriptor is the vector of raw patch intensities. The lecture compares descriptors with the sum of squared differences

$$\mathrm{SSD}(u, v) = \sum_i (u_i - v_i)^2, \tag{34.1}$$

and normalized correlation

$$\rho(u, v) = \frac{\sum_i (u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_j (u_j - \bar{u})^2} \sqrt{\sum_j (v_j - \bar{v})^2}}. \tag{34.2}$$

where

- u, v: two vectorized image patches;
- u_i, v_i: corresponding pixel values;
- $\bar{u}, \bar{v}$: patch means;

- SSD: direct squared intensity difference, not invariant to intensity change;
- ρ: normalized correlation, invariant to independent positive intensity gains and additive offsets when both patches have nonzero variance. A negative gain reverses its sign; a constant patch makes the denominator zero.

The slides additionally list Zero-Mean Normalized SSD (ZNSSD) and Zero-Mean Normalized Sum of Absolute Differences (ZNSAD).

A weakness of raw pixel patches is that small shifts can change the score dramatically. Histograms trade exact spatial order for stability to small geometric perturbations.

34.2 SIFT Descriptor

The lecture's SIFT descriptor uses a 16×16 normalized patch. It divides that patch into a 4×4 grid of cells and builds an eight-bin gradient-orientation histogram in each cell. The result has

$$16 \text{ cells} \times 8 \text{ orientations} = 128 \text{ dimensions}.$$

Figure unavailable

`sift_descriptor.jpg`

Figure 34.1. Source SIFT illustration: a normalized patch is divided into cells, each represented by an orientation histogram.

SIFT normalizes, clips, and then renormalizes the nonzero descriptor vector:

$$u = \frac{d}{\|d\|_2}, \qquad v_i = \min(u_i, 0.2), \qquad \widehat{d} = \frac{v}{\|v\|_2}. \tag{34.3}$$

where

- d: raw nonnegative gradient-histogram descriptor;
- u: initial unit-length descriptor;
- v: descriptor after clipping components at 0.2;
- $\widehat{d}$: final unit-length descriptor. Its components can exceed 0.2 after renormalization; clipping and final normalization are successive operations, not simultaneous bounds.

The lecture attributes SIFT's robustness to two ideas: gradients are less sensitive to illumination changes than raw intensities, and "subdivide and disorder" makes the descriptor tolerant of small shifts while still retaining coarse spatial layout. The slides report robustness to viewpoint changes of roughly 30° out of plane, substantial illumination changes, and real-time implementation. The source also points to David Lowe's SIFT resources at `http://www.cs.ubc.ca/~lowe/keypoints/`.

34.3 Generating Putative Matches

For every feature in one image, generate a short list of candidate features in the other image based on appearance. The lecture presents exhaustive search, approximate nearest-neighbor search, hierarchical structures such as kd-trees and vocabulary trees, and hashing.

A feature-space outlier heuristic compares the nearest-neighbor distance d_1 with the second-nearest distance d_2. Ambiguous features have a relatively high ratio d_1/d_2. The lecture states that a threshold of 0.8 provides good separation.

Chapter summary. Feature descriptors compare normalized local appearance. Raw intensities support SSD or normalized correlation but are shift sensitive. SIFT summarizes gradient orientations over a 4×4 cell grid into a 128-dimensional descriptor, gaining robustness to illumination and small shifts. Putative matches can then be generated by nearest-neighbor search and pruned with the nearest/second-nearest distance ratio.

CHAPTER 35

Robust Alignment from Putative Matches

35.1 The Outlier Problem

Putative feature matches can contain a very high percentage of incorrect correspondences. The lecture considers RANSAC, incremental alignment, the Hough transform, and hashing as strategies for finding a geometric transformation supported by only a subset of all matches.

35.2 RANSAC for Alignment

The alignment version of RANSAC repeats four operations:

1. randomly select a seed group of putative matches;
2. compute the transformation from that seed;
3. find all matches that are inliers to the transformation;
4. if the consensus is sufficiently large, recompute a least-squares transformation from all inliers.

The transformation with the largest inlier set is retained.

Figure unavailable

`alignment_ransac_translation.jpg`

Figure 35.1. Translation example from the alignment lecture: putative matches, one hypothesized translation, and refinement toward an average translation supported by inliers.

The source notes that practical putative-match sets may contain 90% or more outliers, motivating alternatives that impose strong locality constraints.

35.3 Incremental Alignment

Incremental alignment starts from close-by matches and grows a local group. Seed generation identifies triples of neighboring features (i, j, k) in the first image and matching triples (i', j', k') in the second image, with corresponding features required to be putative matches and the matched triple preserving neighborhood proximity.

Beginning from each seed triple, the algorithm repeatedly estimates the alignment transformation, adds other nearby matches consistent with it, and stops when the transformation is no longer consistent or no additional matches can be found.

35.4 Hough-Space Alignment

If features are already scale and rotation invariant, a single match supplies a coarse alignment hypothesis in translation, scale, and orientation. Because one match is unreliable, each putative match votes in a coarsely quantized Hough space and peaks indicate hypotheses supported by several features.

The lecture shows a conventional Canny-to-Hough example as a bridge from edge voting to generalized alignment voting.

Figure unavailable

`alignment_hough_edges.jpg`

Figure 35.2. Source Hough example: Canny edge evidence votes in parameter space; peaks are converted back into detected image edges.

In the **generalized Hough transform**, displacements are indexed by matched local patterns or "visual codewords" rather than only by gradient orientation.

35.5 Lowe's Hough-Space Verification

The lecture describes Lowe's system in two phases. During training, each model feature stores model location, scale, and orientation relative to its normalized feature frame. During testing, every model–test feature match votes in a four-dimensional Hough space. The stated bin sizes are 30° for orientation, a factor of 2 for scale, and 0.25 times image size for location, with votes placed into the two closest bins in each dimension.

Bins receiving at least three votes undergo geometric verification: estimate a least-squares affine transformation, apply stricter residual thresholds, and search for additional features agreeing with the alignment.

> **Chapter summary.** Robust alignment must recover a transformation from correspondence sets dominated by outliers. RANSAC uses random seed groups and consensus; incremental alignment exploits local neighborhoods; Hough alignment lets individual invariant matches vote for coarse transformation parameters. Geometric verification then refines promising hypotheses and tests them against additional matches.

CHAPTER 36

Homographies and Panorama Stitching

36.1 Beyond Affine Transformations

The lecture asks which transformation relates two views of a planar surface or two images from cameras sharing the same center. The answer is a **homography**, or plane projective transformation, which can map between nondegenerate quadrilaterals (with no three vertices collinear).

A 2D point is converted to homogeneous coordinates as

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

while a homogeneous point $[x, y, w]^T$ converts back to Euclidean coordinates as $(x/w, y/w)$ when $w \neq 0$; $w = 0$ represents a point at infinity.

The homography equation is

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \tag{36.1}$$

where

- (x, y): source image coordinates;
- (x', y'): destination image coordinates;
- $H = [h_{ij}]$: 3×3 homography matrix;
- λ: arbitrary homogeneous scale.

A homography matrix must be nonsingular. It has nine entries but only eight degrees of freedom because overall nonzero scale is arbitrary. Estimating it requires at least four point correspondences in general position, with no three collinear in either image.

36.2 Panorama Recognition and Stitching

The panorama pipeline in the lecture automatically determines which images from a camera memory card belong together and then stitches each connected group. Its high-level stages are:

1. estimate pairwise homographies robustly with RANSAC;
2. find connected sets of mutually alignable images;
3. warp, stitch, and blend the images in each connected set.

Figure unavailable

`panorama_results.jpg`

Figure 36.1. Final panorama-stitching results shown in the alignment lecture after homography estimation, grouping, warping, and blending.

36.3 Pyramid Blending

Simple seams remain visible when aligned images have different exposure or local structure. The lecture introduces pyramid blending: build a Laplacian pyramid and blend each frequency level separately. The source cites P. J. Burt and E. H. Adelson, "A Multiresolution Spline with Application to Image Mosaics," *ACM Transactions on Graphics*, 2(4), October 1983, pp. 217–236, and gives `http://persci.mit.edu/pub_pdfs/spline83.pdf`.

Figure unavailable

pyramid_blending.jpg

Figure 36.2. Pyramid-blending example from the lecture, illustrating multiresolution transition between source images.

36.4 Alpha Blending

The lecture encodes a blend-weighted image sample as

$$I(x, y) = (\alpha R, \alpha G, \alpha B, \alpha). \tag{36.2}$$

If three images contribute to a pixel p, the displayed color is

$$C(p) = \frac{(\alpha_1 R_1, \alpha_1 G_1, \alpha_1 B_1) + (\alpha_2 R_2, \alpha_2 G_2, \alpha_2 B_2) + (\alpha_3 R_3, \alpha_3 G_3, \alpha_3 B_3)}{\alpha_1 + \alpha_2 + \alpha_3}. \tag{36.3}$$

where

- (R_i, G_i, B_i): color from image i at the pixel;
- α_i: blend weight for image i;
- premultiplication: each RGB value is multiplied by its weight before accumulation;
- the denominator: normalizes accumulated color by total weight.

Implementation is described in two steps: accumulate premultiplied RGB values and alpha weights, then divide the accumulated RGB value at each pixel by its accumulated alpha wherever that sum is positive. A pixel with zero total weight has no contributing image and must be masked or assigned a background value. The source optionally refers readers to J. F. Blinn's 1994 *IEEE Computer Graphics and Applications* discussion for further alpha-blending details.

36.5 Gain Compensation

Exposure mismatch can produce visible artifacts even after geometric alignment. The lecture's simple gain-compensation method computes the average RGB intensity of each image in the overlap and normalizes image intensities according to the ratio of those averages.

Figure unavailable

gain_compensation.jpg

Figure 36.3. Gain-compensation example from the lecture: overlap statistics are used to reduce visible intensity discontinuities between aligned images.

36.6 Design Choices in Alignment Applications

The final slides summarize several coupled choices:

- **geometric model**: balance correctness against robustness and efficiency;
- **descriptor**: rich natural imagery supports high-dimensional patch descriptors such as SIFT, whereas impoverished imagery such as star fields may require invariant geometric descriptors built from tuples of point features;

- **putative-match search**: brute force is reasonable for small one-time problems such as panorama stitching; large model databases favor indexing or hashing and feature-space pruning heuristics;
- **hypothesis generation**: RANSAC is appropriate when the inlier ratio is relatively large, while very small inlier ratios favor locality constraints or Hough voting;
- **verification**: consensus-set size and residual tolerance depend on expected inlier ratio and model accuracy; the model can be refined and checked densely after a plausible hypothesis is found.

Chapter summary. Homographies extend affine alignment to planar projective transformations and have eight degrees of freedom. Panorama stitching estimates homographies robustly, groups compatible images, and blends warped results. Pyramid and alpha blending address seams, while gain compensation addresses exposure differences. Practical alignment quality depends jointly on the geometric model, descriptor, matching strategy, hypothesis generator, and verification criteria.

Lecture 9 Quick Review

Models and descriptors

1. Alignment fits a transformation T between corresponding features in two images.
2. Similarity, affine, and projective models trade flexibility against robustness and estimation cost.
3. An affine transformation has six unknowns; three point matches with non-collinear source points are the minimum for a unique fit.
4. Raw patch descriptors can use SSD or normalized correlation; histogram-based descriptors are more robust to small shifts.
5. SIFT uses 4×4 cells and 8 orientation bins per cell, yielding 128 dimensions.
6. The nearest/second-nearest distance ratio helps reject ambiguous putative matches; the lecture uses 0.8 as a useful threshold.

Robust alignment and panoramas

7. RANSAC hypothesizes transformations from seed matches and verifies them by consensus.
8. Incremental alignment exploits neighboring feature triples and grows locally consistent groups.
9. Hough alignment lets invariant matches vote for translation, scale, orientation, and related parameters.
10. A homography is a 3×3 projective transformation with eight degrees of freedom up to scale.
11. Panorama stitching estimates homographies, finds connected image sets, then warps and blends them.
12. Pyramid blending handles multiscale seams; alpha blending normalizes weighted RGB accumulation; gain compensation reduces exposure artifacts.